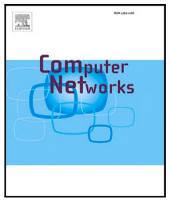
Computer Networks 243 (2024) 110310



Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet





Adaptive overlay selection at the SD-WAN edges: A reinforcement learning approach with networked agents

Alessio Botta, Roberto Canonico, Annalisa Navarro *, Giovanni Stanco, Giorgio Ventre

DIETI, Università degli Studi di Napoli Federico II, Via Claudio 21, 80125, Napoli, Italy

ARTICLE INFO

Keywords:
SD-WAN
SDN
Multi-agent reinforcement learning
Traffic engineering

ABSTRACT

Software-Defined Wide Area Networks (SD-WANs) have emerged as a promising solution to address the connectivity demands of modern distributed enterprises. However, the effective application of the Software Defined Networking (SDN) paradigm in such broad and dynamic environments remains a significant challenge. In this paper, we present two novel contributions. First, we design a decentralized control plane for SD-WANs that leverages edge-based network monitoring and overlays' configuration. Then we present a Reinforcement Learning-based orchestration plane that leverages local information for the enforcement of SD-WAN policies. Since traditional approaches suffer either a lack of scalability due to the problem's complexity or suboptimal performance due to isolated decision-making, the proposed approach leverages a cooperative Multi-Agent Reinforcement Learning framework. Our novel cooperative approach is based on per-site agents that exchange a small amount of information to enhance performance while preserving scalability. To validate the efficacy of our proposed approach, we conducted an extensive experimental evaluation considering diverse SD-WAN scenarios. Results show that our framework is able to satisfy global network policies for a multi-site SD-WAN with different QoS requirements and cost constraints.

1. Introduction

Software Defined Wide Area Network (SD-WAN) is a rapidly emerging technology that offers a versatile solution for adapting networks on both temporal and geographic scales. Its primary objective is to ensure secure and reliable communication between numerous distributed sites spanning over a vast geographical region, interconnected through multiple overlays [1]. By capitalizing on real-time monitoring and centralized control, SD-WAN enables swift network reconfiguration to meet the Quality of Service (QoS) requirements of various applications. While Software Defined Networking (SDN) has undeniably revolutionized traditional traffic management in Local Area Networks (LANs) [2], it faces its own set of complexities in Wide Area Networks (WANs) [3].

While SD-WAN pledges to deliver Quality of Service (QoS) and high availability for users and applications, achieving this in practice is not straightforward. SD-WANs can use costly dedicated lines or protocols offering QoS guarantees, such as Multiprotocol Label Switching (MPLS), as well as the best-effort public Internet [4]. Consequently, the main challenge of SD-WAN lies in orchestrating traffic, by leveraging all the available overlay connections to ensure QoS levels comparable to those provided by dedicated lines.

In SD-WAN solutions, QoS requirements are typically implemented through network policies. A network policy can be used to affect the forwarding of data traffic in the overlays data plane to reach specific business outcomes in an automatic way. Consider a scenario where a policy is defined, specifying that latency-sensitive traffic should not encounter delays beyond 500 ms. In such cases, any deviation from this threshold should prompt an immediate rerouting of the traffic to an alternative WAN transport that ensures the QoS requirement is satisfied.

Network QoS policies are enforced in SD-WANs through the joint action of control and orchestration plane.

The conventional fully centralized control plane, which possesses global knowledge and oversees the entire network, proves itself to be intricate and risky since it becomes a single point of failure. To address this, we leverage a distributed controller architecture, in which each controller maintains a local view and manages only a subset of the overall network architecture. This approach improves scalability and reduces the impact of failures [5].

While at the state of the art, the most promising solution for the realization of network policies in the orchestration plane is Reinforcement Learning (RL), this approach suffers scalability limitations. In [6]

* Corresponding author.
E-mail addresses: alessio.botta@unina.it (A. Botta), roberto.canonico@unina.it (R. Canonico), annalisa.navarro@unina.it (A. Navarro), giovanni.stanco@unina.it (G. Stanco), giorgio.ventre@unina.it (G. Ventre).

https://doi.org/10.1016/j.comnet.2024.110310
Received 10 October 2023; Received in revised form 16 February 2024; Accepted 4 March 2024
Available online 6 March 2024
1389-1286/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Table 1
State-of-the-art solutions for traffic engineering in SD-WAN. (TLM: Transport Layer Monitoring; INT: In-band Telemetry).

Ref	Target	Problem	Solution	Monitoring	Topology	Evaluation
[6]	QoS and Cost	Overlay Selection (Dynamic)	MARL	Active (Probing)	Full-mesh	Emulation
[7]	Bandwidth	Failure Recovery	Greedy	Active (Probing)	Both	Testbed
[8]	Transit Cost	Overlay Selection (Static)	Deterministic	-	-	Testbed
[9]	QoS	Rerouting	Threshold-based	Active (Probing)	Hub and Spokes	Testbed
[10]	QoS	QoS-aware Load Balancing	DRL + RCPO	Passive (INT)	Hub and Spokes	Theoretical
[11]	Availability	Overlay Selection (Dynamic)	DRL	Active (Probing)	Hub and Spokes	Static Dataset
[12]	Bandwidth	Failure Recovery	MPTCP Proxies	Passive (TLM)	Hub and Spokes	Testbed
[13]	QoS and Cost	Overlay Selection (Dynamic)	RL	Active (Probing)	Full-mesh	Emulation
[14]	Bandwidth	Intent Based Routing	Local Search	Passive (INT)	Hub and Spokes	Emulation
[15]	Latency	Failure Recovery	Threshold-based	Passive (TLM)	Hub and Spokes	Real Testbed
[16]	SLA satisfaction	Load Balancing	Graph Convolutional RL	Passive (INT)	Hub and Spokes	Emulation
[17]	QoS and Cost	Traffic Scheduling	Lyapunov Optimization	-	Hub and Spokes	Trace-based
[18]	SLA satisfaction	Load Balancing	Local Search	Proprietary Algorithm	Hub and Spokes	Simulation
[19]	SLA Satisfaction	Load Balancing	Graph Convolutional RL	Proprietary Algorithm	Hub and Spokes	Simulation
Our work	QoS and Cost	Overlay Selection (Dynamic)	Cooperative MARL	Active (Probing)	Both	Emulation

we addressed the scalability issue by using decentralized Multi Agent Reinforcement Learning (MARL) that helps reduce complexity through parallel decision-making. Unfortunately, the employed approach is *fully distributed*, which implies that the agents take completely independent actions, without considering their mutual influence.

In this paper, we show that, while this design choice simplifies the evaluation process, it does not capture the interactions of traffic in the underlay [7]. To address this, we propose a collaborative version of MARL, designated as MARL with Networked Agents (MARL-NA), in which the agents actively interact with each other and exchange rewards. By sharing rewards, these cooperative agents collectively optimize the decision-making process and achieve improved overall network performance. Furthermore, we design and implement a decentralized control plane consisting of network monitoring and configuration functions implemented at Edge Sites based on Customer Premise Equipments (CPEs) and network controllers.

We conducted extensive experiments within an emulated environment, simulating diverse WAN scenarios with multiple SD-WAN sites, varying QoS requirements, and different network policies. The results demonstrate that our approach excels in reconciling conflicting policies and achieving global objectives while fostering cooperation among distinct agents. Our approach consistently exhibits fewer Service Level Agreements (SLA) violations in comparison to baseline methods, all while maintaining low latency and transit costs.

The paper follows this structure: Section 2 provides an overview of related work, covering distributed controller architectures for WANs (Section 2.1), traffic engineering, orchestration solutions within the context of SD-WAN (Section 2.2), and diverse MARL methodologies (Section 2.3). Section 3 introduces the SD-WAN use case and outlines the problem statement. In Section 4, we detail our proposed framework, including the design of the decentralized control plane with network monitoring and configuration functions (Section 4.1) and the Cooperative MARL solution within the orchestration plane (Section 4.2). Section 5 delves into the experimental evaluation and presents our results. Finally, Section 6 draws conclusions.

2. Related work

2.1. Controller architectures

A single controller architecture for WANs can have several limitations and problems, which can impact network performance, scalability, and reliability [5]. A single controller may become a single point of failure, disrupting the operation of the entire network when experiencing failure or downtime. A single controller also experiences scalability challenges, struggling to handle management and monitoring tasks in a WAN, which is an environment always growing in scale and complexity. Furthermore, since all network decisions and policies are made by the same entity, routing decisions may be affected by a delay, impacting the real-time applications and services.

Several research studies have highlighted the importance of implementing a distributed control plane in an SD-WAN architecture. This is because a centralized controller design is inadequate in achieving the desired levels of resiliency and scalability. These studies specifically emphasize the strategic deployment of multiple controllers across WAN topologies to enhance both control plane latency and fault tolerance. [20], or utilizing load-sharing mechanisms to distribute data plane switches among multiple controller instances [21]. While the control plane is physically distributed, many applications still require a global view of the network. In these use cases, any update tracked by a controller has to be forwarded to all other controller instances. However, achieving such consistency while preserving performance poses a complex challenge.

In [22], authors classified distributed controller architectures based on their physical organization. The first category involves a *hierarchical control plane*, where the network control logic is divided into multiple layers. The second category is a *flat control plane*, where network control is horizontally partitioned to manage multiple areas. In hierarchical controllers, each layer serves specific functions. Typically, lower-layer controllers manage local events and serve switches, while the top layer offers a global perspective and facilitates interoperability between lower-layer, area-specific controllers. Notably, lower-layer controllers do not establish direct connections with each other; instead, each local controller utilizes the services of the upper-layer controller for inter-domain connectivity. In a flat SDN control plane, the network is

segmented into multiple areas, each governed by a local controller. This organizational structure enhances resilience and reduces control latency. To maintain a global view, all controllers within a flat cluster remain interconnected through an eastbound interface, facilitating the sharing and reception of network state information among controllers.

A specific instance of a flat architecture is the *fully distributed* model. In this configuration, controllers do not relay all updates to each other. Instead, locally generated data stays within the instance where it originated and is shared with other instances only when necessary. In this approach, direct communications between controllers are initiated only when there is a need to exchange information for establishing inter-site services. This method of connecting controllers enhances robustness in the face of connectivity problems, as any network disconnection or node failure only affects a subset of the infrastructure. An example of this kind of architecture is DISCO [23]: its architecture involves multiple distributed controller instances, each responsible for controlling a single SDN domain. To facilitate communication and information sharing between controller instances, a lightweight channel is established between them to exchange summarized domain information. Similarly, the Edge Controlled Hybrid Software-Defined Networking (EC-hSDN) [24] approach is based on the concept of segregating the edge network from the core network. In EC-hSDN, programmable nodes are deployed at the network's edge, and an SDN controller manages the traffic that enters and exits the organization.

With respect to the state of the art, we embrace the EC-hSDN approach, employing a Customer Premises Equipment (CPE) at every SD-WAN site. Each CPE is responsible for the traffic entering and exiting the site. To ensure scalability, the controller architecture follows a fully decentralized approach, with each controller being responsible for a single CPE. This architecture is designed to be flat, without a central controller and hierarchy levels, to ensure robustness and low latency. For the communication among controllers, lightweight channels are established to facilitate the exchange of monitoring information.

2.2. Traffic engineering

In this section, state of the art regarding Traffic Engineering (TE) in SD-WAN is presented.

A TE SD-WAN application was introduced in [15] that comprises a monitoring system that issues alerts upon detecting performance degradation in the current overlay and a module responsible for switching overlays whenever an alert is received. The application's effectiveness was evaluated in two testbeds — a real-world two-site testbed in a metropolitan city and an emulated one with four sites. The results demonstrated an enhancement in service availability and a reduction in delay for time-sensitive traffic. A different method presented in [14] consists of a semi-distributed, intent-based system to enhance the implementation of routing policies. The experimental setting included a main headquarters location and three branch locations connected in a hub-and-spoke network configuration. Different overlays are chosen for applications depending on specified criteria or network objectives, such as reducing latency, congestion, or expenses. The system utilizes a local minimum search algorithm in conjunction with a traffic prediction module to determine the most effective set of overlays.

A recent advancement proposed a Multi-Path Transmission Control Protocol (MPTCP) that is aware of the SD-WAN technology [12]. This approach has the objective of optimizing the utilization of numerous, diverse WAN transports through aggregation while also ensuring quick recovery from failures. The efficacy of this method was proven in a simulated test environment consisting of 2 branch sites and 5 tunnels, as well as in a real network with 3 branch sites and 2 tunnels. Authors in [11] proposed RL techniques in SD-WAN for the implementation of network policies. While in a preliminary study [9], they used a simple threshold-based algorithm, in this extended version, they conducted experiments using a set of Deep Reinforcement Learning (DRL) algorithms. They evaluated them on a dataset of delay time series from an SD-WAN environment consisting of two interconnected sites and multiple overlays between them. Results indicated that the Deep Q-Learning algorithm exhibited the fastest convergence time and demonstrated the effectiveness of RL in achieving the pre-set policy constraints.

The authors of [10] utilized DRL to calculate flow-splitting ratios. This was done to distribute traffic across several paths in order to decrease the delay while also ensuring that capacity limits are met. The method was assessed in an SD-WAN scenario that consisted of a central headquarters site and three branch sites connected by both MPLS and the Internet. The proposed approach effectively achieves minimal end-to-end delay and mitigates connection capacity violations. The authors assess their method by modelling the environment through queuing theory. The researchers in [16] introduced MARL for scheduling traffic flows, taking into account the respective traffic category. The technique attempted to fulfill the throughput or delay requirements for each flow. The proposed methodology, assessed in a hub-and-spokes configuration, showcased that a hybrid MARL approach adeptly caters to traffic demands and outperforms a completely decentralized strategy.

The authors of [17] presented Grandet, a framework designed to schedule SD-WAN flows for online traffic management. The primary objectives of this framework are to ensure QoS and reduce transit costs. This approach utilizes a combination of neural networks to forecast flow demand intervals and Service Level Agreement (SLA) parameters. It subsequently formulates an optimization problem using Lyapunov control. Empirical assessments using actual traffic data from an ISP have shown that their approach effectively lowers transmission expenses and enhances QoS when compared to baselines. The methodology presented in [7] introduces a mechanism for updating routes to optimize the selection of SD-WAN links. It notably focuses on problems related to inefficient use of available network capacity and the lack of a full view of the entire network. The method employs active tomography to deduce the features of the WAN and utilizes a greedy solution strategy to tackle the minimum-cost problem, which is known for its NP-hard complexity. The effectiveness of the methodology is assessed by conducting practical tests on real network topologies to evaluate the number of flow disruptions.

The authors of [8] introduced a Dynamic Traffic Management (DTM) strategy that aims to efficiently distribute new flows across links in order to minimize transit costs. This technique is specifically designed for situations where the cost tariff is determined by the amount of traffic and the 95th percentile measure. Their findings demonstrate a significant decrease of 30% in the expenses associated with routing interdomain traffic.

The paper [18] presented a model for optimizing global QoS policies. This model allows for the dynamic adjustment of application rate constraints in response to fluctuations in network conditions. The model employs a cross-traffic estimator to assess the bandwidth of overlay links, by comparing centralized and distributed local search algorithms. The simulation findings demonstrate a 40% enhancement in SLA satisfaction when compared to static policies. The researchers in [19] utilize a Graph Convolutional Deep Reinforcement Learning framework to enhance the overall performance of the network. They show enhancements in throughput and end-to-end delay when compared to conventional methods.

In [13], we presented an RL-based dynamic routing framework for SD-WAN that is based on network measurements to direct traffic to one of two possible overlays linking two branch locations of a corporation. By adjusting to very unpredictable network conditions, this architecture aims to achieve performance and transit cost objectives. We ran tests in a simulated setting and found that, in comparison to a naive method, there were fewer policy violations and lower costs. We explored the scalability of this technique in [6], expanding upon earlier work by considering a more sophisticated scenario with numerous locations and overlays. We proposed a fully distributed Multi-Agent Reinforcement Learning framework (MARL), showing that it manages to improve

Service Uptime and reduce latency while also keeping complexity manageable.

Table 1 summarizes the state-of-art solutions for TE in the SD-WAN environment, particularly reporting the specific objective to be achieved (e.g. maximizing bandwidth utilization, guaranteeing QoS, minimizing transit costs, etc.), the main problem involved (e.g. dynamic overlay selection, failure recovery, traffic scheduling), the proposed solution (with Reinforcement Learning being the most popular approach), the monitoring technique involved (passive or active), the evaluation methodology (e.g simulations, testbeds ...), and the proposed SD-WAN configuration (hub-and-spokes or full-mesh).

In this paper, we specifically follow a dynamic overlay selection approach to meet global network policies regarding QoS and cost. While in [6], we leverage a fully decentralized Reinforcement Learning algorithm, in this paper, we present a cooperative approach to minimize overall policy violations and evaluate its effectiveness compared to the fully decentralized MARL approach we already described in [6]. Additionally, we rely on an active monitoring approach rooted in a decentralized architecture that is suitable for both hub-and-spokes and full mesh network topologies. Eventually, we evaluate our approach by packet-level simulations.

2.3. Multi agent reinforcement learning

MARL can be categorized into two classes: centralized MARL and decentralized MARL. Centralized MARL involves a single central entity or coordinator that gathers information from all agents and collectively learns policies for the entire group. This approach reduces the multi-agent problem to a single-agent one and suffers from exponential dependence as it usually needs to search the joint action space exhaustively [25]. On the other hand, decentralized MARL involves agents learning individual policies without a central coordinator. Each agent interacts with its environment and learns its policy based on its observations and rewards.

In the fully decentralized MARL setting, no central controller exists, each agent operates independently using local observations and the control execution is local. This architecture offers several advantages, including scalability to a large population of agents and robustness to malicious attacks. Besides, since agents in this setting make decisions based on their local context without coordination or central aggregation of data, the fully decentralized approach often fails to meet global objectives [26].

In recent years, decentralized MARL with Networked Agents (MARL-NA) has gained significant attention, introducing a unique perspective to multi-agent systems [26,27]. Each agent makes individual decisions according to local observations and information shared by its neighboring agents through a communication network. The networked structure is represented by an undirected graph, where nodes correspond to agents and edges indicate communication channels between agents.

The concept of reward sharing has emerged as an essential aspect of networked MARL. Authors in [28] suggest that agents can share rewards with their neighbors, promoting cooperation by encouraging agents to consider the consequences of their actions on others. Reward sharing is limited to neighboring agents, as actions of neighbors directly impact an agent's reward, while actions of distant agents affect returns indirectly through state distribution changes.

To wrap up, fully cooperative multi-agent systems are computationally intensive due to the combinatorial explosion in joint action–state spaces [26], while fully decentralized MARL lacks the cooperation needed to reach common objectives. Therefore, aiming to achieve both a low complexity level and a certain degree of cooperation among agents, we leverage MARL-NA with agents deciding independently the best action but at the same time considering information exchanged with each other to reach a common goal.

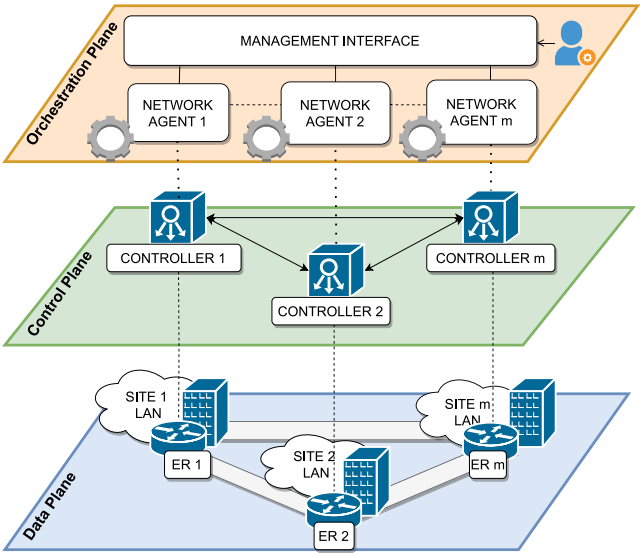


Fig. 1. SD-WAN reference architecture.

3. Reference architecture and problem statement

3.1. Reference architecture

SD-WAN architecture is logically separated into three planes as in [29]: data plane, control plane, and orchestration plane. SD-WAN's *data plane* establishes connections through private and public IP/WAN infrastructures, simplifying communication across distant sites and cloud services. It achieves this by creating a software-managed overlay network atop the existing physical underlay infrastructure. While the overlay is consistent, the underlay WAN infrastructure is often diverse, ranging from single uplinks like MPLS to combinations of technologies like DSL, 4G/5G, and leased lines. The *control plane*'s main entity is the controller, positioned at various sites, including branches, headquarters, data centers, or the cloud. This controller manages the configuration and monitoring of the ERs through the southbound API. Controllers communication happens through the East-West API. To program the SD-WAN service for the orchestrator, the northbound API is employed, utilizing RestAPI for communication. The *orchestration plane* of the SD-WAN system can be situated either within the on-premises IT infrastructure or in the cloud. At the core of this layer lies the orchestrator, a key component dedicated to integrating and orchestrating the entire SD-WAN solution. This orchestrator can function as a standalone entity or be integrated into the controller, possibly comprising multiple sub-entities. It plays a crucial role in enforcing centralized and unified policy management, configuration management, troubleshooting, monitoring, and analytics across the network.

As depicted in Fig. 1, we employ a flat control plane architecture, with each controller responsible for the ER at a specific geographical site. The controllers exchange monitoring information among themselves. In the orchestration plane, there are several agents, each accountable for a particular site and communicating with the controller of that site and with each other when necessary. Network agents receive requirements from the management interface used by the network administrator to enforce network policies.

Fig. 2 shows the reference SD-WAN data plane into consideration: M geographically distant sites S (e.g. enterprise or data center sites) are interconnected through diverse overlays by means of M Edge Routers ER. ERs collect traffic flows from the individual sites and forward them to the destination through the available overlays. O_{ij}

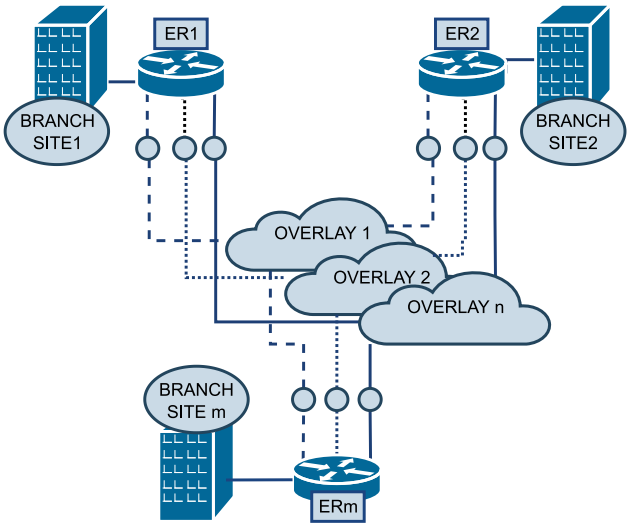


Fig. 2. A typical SD-WAN data plane: m sites connected through n overlays.

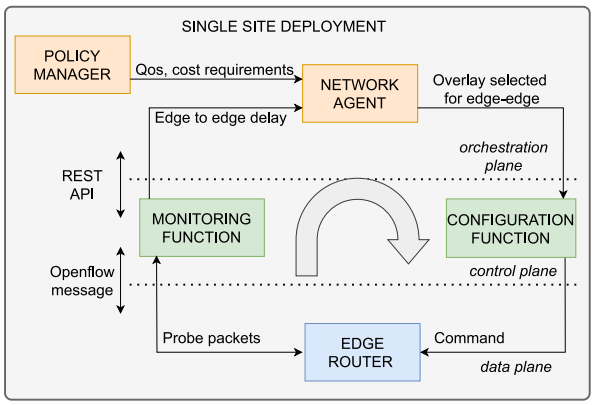


Fig. 3. Feedback-based control loop for network policy implementation (monitor, analyze, decide, act). The Data Plane is in blue, the Control Plane is in green, and the Orchestration Plane is in orange. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

is the set of overlays o_{ijk} between the couple of sites s_i and s_j for all i and for all j in M with $i \neq j$ with cardinality N_{ij}. Overlays can have different characteristics (e.g., MPLS, Internet, ...), and the corresponding connectivity may be provided by different Internet Service Providers (ISPs). For this reason, in most scenarios, there is no control over the overlays and their intra-domain routing. The only network configuration that can be changed is the one of ERs [30,31]. Tunnels connecting diverse sites can be established through a range of protocols (e.g. Generic Routing Encapsulation (GRE), Internet Protocol Security (IPsec), and Virtual Extensible LAN (VxLAN)). Importantly, our proposed framework remains agnostic to the specific type of encapsulation implemented, ensuring flexibility and adaptability across various deployment scenarios.

3.2. Problem statement

ERs are integral components of the enterprise network and can receive direct instructions from a controller to execute certain operations on traffic. Instances of such operations encompass the utilization of per-flow or per-packet load balancing methods and the utilization of numerous overlay networks to aggregate bandwidth. Likewise, it is common for ERs to replicate packets in order to guarantee dependable transmission or for the purpose of error correction. Furthermore, they have the ability to dynamically change routes for flows that are not achieving the required constraints in terms of QoS guarantees [32]. Specifically, this paper tackles the challenge of dynamically responding to performance degradation in overlays by switching to an alternative overlay for routing traffic when the current performance is insufficient. To enable this near real-time control, we rely on feedback from the network to assess its current conditions. In particular, we leverage the measurement of d_{ijk} – i.e. the delay of the kth overlay between s_i and s_j at time t – as feedback.

SD-WAN solutions empower enterprise network administrators to establish policies aligning with either QoS needs or budget constraints. Continuous network measurements are compared against the requirements of applications or the network itself, to enable the selection of overlays that can effectively route traffic satisfying those specified constraints. In this paper, we consider two policies, both imposing an upper limit to the edge-to-edge delay. The first policy (Policy A) only considers delay, while the other one (Policy B) jointly considers delay and transit cost. Given an ordered pair of communicating sites $S_{ij} = (s_i, s_j)$ and a set of overlays O_{ij} among them, we assume that a network administrator selects a delay threshold T_{ij} as the maximum allowed latency for the communication from s_i to s_j. Two different policies can be precisely defined as follows.

1. **Policy A:** At each timestamp t for each pair of communicating sites s_i and s_j, select o_{ijk} from O_{ij} that guarantees $d_{ijkt} < T_{ij}$.
2. **Policy B:** At each timestamp t for each pair of sites s_i and s_j, select o_{ijk} from O_{ij} that guarantees $d_{ijkt} < T_{ij}$ and has the minimum cost per traffic volume among all overlays with $d_{ijkt} < T_{ij}$.

We define two objectives concerning the policies. The former objective is to minimize the SLA violations, while the latter objective is to minimize the policy violations. A rigorous definition of the SLA violation and Policy A and Policy B violation is in the following.

- SLA is considered violated at time t if the selected overlay o_{ijk} does not guarantee $d_{ijk} < T_{ij}$.
- Policy A is considered violated at time t if the selected overlay o_{ijk} does not guarantee $d_{ijkt} < T_{ij}$ and there exists another overlay o_{il} with $l \neq k$ such that $d_{ijlt} < T_{ij}$.
- Policy B is considered violated at time t if at least one of the following conditions is false: (i) the selected o_{ijk} does not guarantee $d_{ijkt} < T_{ij}$, and there exists another overlay o_{il} with $l \neq k$ such that $d_{ijlt} < T_{ij}$; (ii) $d_{ijkt} < T_{ij}$, but there exists another overlay o_{il} for which $d_{ijlt} < T_{ij}$ and its cost per traffic volume is lower than that of o_{ijk}.

4. Framework

In order to effectively implement in-practice network policies, we leverage a feedback-based control loop involving different phases: monitoring, data analytics, decision-making, and action. In the *monitoring* phase, network performance metrics are continuously measured. In particular, we measure d_{ijk}, i.e. the latency of the k_{th} overlay between site i and site j for each i and j belonging to M with $i \neq j$, using a probing mechanism. In the *analysis* and *decision-making* phases, the collected data are processed to derive the network's current state with respect to the set policy. A Reinforcement Learning decision engine is employed to decide o_{ij}, i.e. the overlay selected for sending traffic from site i to site j for all i and j in M with $i \neq j$. In the *action* phase, new configurations based on the decisions made in the previous step are sent to the networking devices.

In Fig. 3 the implementation of the control loop in a single site architecture is shown: the monitoring and action phases are implemented

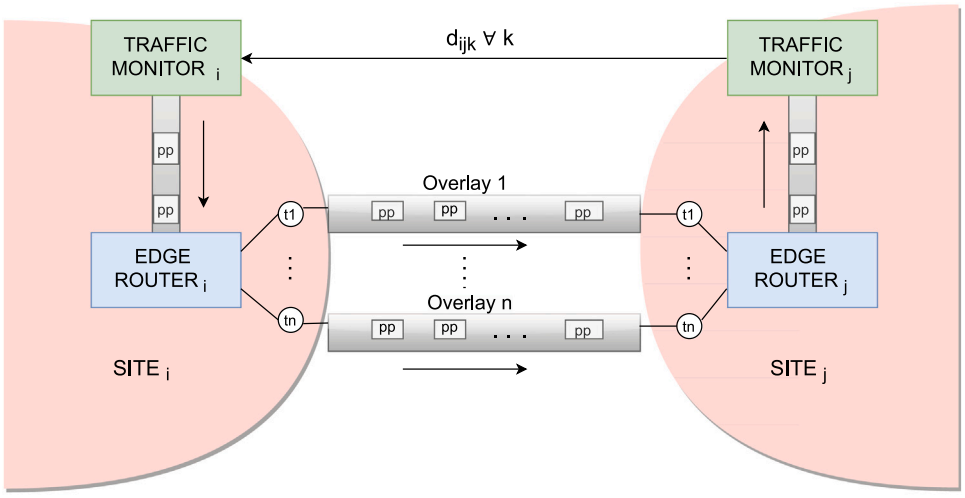


Fig. 4. Decentralized Monitoring System: the measurement of overlay latency is accomplished by dispatching probe packets (pp) from a traffic monitoring entity to traverse each of the available overlays.

in the control and data planes. The employed monitoring function in the control plane continuously sends probe packets via the ERs to infer the delay on the overlays, as we will discuss later on. The configuration function in the control plane is responsible for the action phase, translating the decision made by the orchestration plane into a configuration for the ERs, specifically by instructing ERs to forward the selected flow through a specific overlay. In the orchestration plane lies a decision engine based on the RL paradigm – that will be discussed in Section 4.2 – that takes in the requirements in terms of the maximum allowed latency for SD-WAN traffic, as well as delay measures made by the control plane, and outputs a decision on the overlay selected to forward SD-WAN traffic.

4.1. Control plane

The two main components of the control plane are the configuration function and the monitoring function. The configuration function waits for requests of configuration changes by the orchestration plane and translates them into actual commands to send to the data plane. Specifically, each controller C_i is responsible for updating ER_i's routing table. Changing rules in the routing table means changing the overlay selected to forward traffic toward another SD-WAN site. The monitoring function is more complex and is described in the following.

4.1.1. Monitoring function

To ensure adaptive real-time adjustments to current conditions, precise measurements with the right level of granularity are needed. However, network monitoring in SD-WAN is non-trivial. SD-WAN paradigm mainly relies on public Internet connections offered by telco operators. Internal network details, routing policies, and link statuses of these WAN connections are unknown, and control over WAN links and intermediary devices is limited. There is a need for a monitoring mechanism that perceives the network between edge sites as a black box, measuring WAN path delay from an external standpoint. Traditionally, there are two kinds of network monitoring: active monitoring, which introduces supplementary traffic to measure network conditions, and passive monitoring, which performs the measurement based on the existing traffic. Active monitoring is generally more accurate and works also in the absence of real network traffic, with the traffic generated for monitoring being negligibly small (a few Kbps).

The method employed to ascertain WAN connection latency aligns with the approach in [13] that follows the active monitoring paradigm. To infer the delay of the path between a couple of sites, probes are sent to traverse it. Specifically, given a couple of sites s_i and s_j, the centralized controller C injects probe packets via ER_i destined to s_j. ER_j is pre-provisioned with a rule that matches some field of the probe packet to send it back to the controller C. This allows the controller that receives the probe back to measure the delay of the WAN connection as the time taken by the probe to come back. Regrettably, this approach is rooted in centralized architecture with a single controller, that proves unviable for large network topologies. The single controller responsible for handling all probe packets becomes overwhelmed, compromising the precision of delay measurements [33]. To address this, we devised a network monitoring function that leverages components located in the different controllers. According to the architecture described in Section 3, each controller C_i controls the corresponding site s_i and is placed within s_i in the proximity of the corresponding ER_i.

The process is depicted in Fig. 4: the controller C_i injects a probe packet via ER_i, that puts the timestamp corresponding to the current time in the packet and forwards it to ER_j via all available overlays. The egress ER_j, once it has received the probe, does not forward it back to a central controller C or the source controller C_i, but instead forwards it to its own controller C_j. Once the probe is received, C_j measures the time difference between the current time and the timestamp inside the packet to infer the delay of the WAN connection. This requires the assumption that the controllers are synched.

The fact that the time difference is computed by the destination controller and not by the source controller enables the computation of the one-way delay from the site s_i to s_j (i.e., d_{ijk}). If the probe packets were sent back to the source controller, they would have traversed the network in the reverse direction. The delay computed at the source would then be the round-trip time (RTT) (i.e., $d_{ijk} + d_{jik}$) that is not an accurate measurement in the case of asymmetric link conditions, as WAN paths are often full-duplex, meaning that the one-way delay in one direction is different from the one-way delay in the opposite direction. In the final step, C_j sends its measurement to the C_i controller, as controller C_i should make use of the C_j measurement (d_{ijk}) to decide o_{ij}, and vice versa. C_j should make use of the C_i measurement (d_{ji}) to appropriately choose o_{ji} and take into account the asymmetry of the network paths.

4.1.2. Control plane implementation

Each controller is multithreaded, with each thread working in a continuous loop and responding to events. The main network functions deployed in the controllers are configuration measurement and probing functions, while the others are dedicated to handling communication with other architectural components (i.e. the northbound API handles the interactions with the orchestration plane, the southbound API the

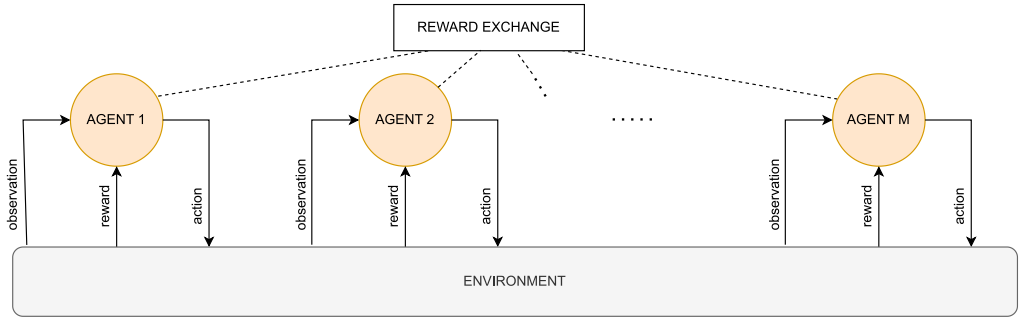


Fig. 5. Multi Agent Reinforcement Learning With Networked Agents: neighboring agents exchange their rewards to enable Cooperation.

interactions with ERs, and the EastWest API the interactions with other controllers). The components involved in the Control Plane are described in the following.

1. *Configuration Function*: The configuration function awaits the decision made by agent A_i. The decision denoted as o_{ij}, represents the overlay to be used for transferring traffic to each other site j belonging to the subset of sites that communicate with s_i. Upon receiving this decision, if there is a change from the previous choice for any site j, the configuration of ER_i is updated by sending a command that modifies the route for site j, overwriting the previous configuration.
2. *Monitoring Function*: The monitoring function is distributed. Thus, each controller has two subfunctions: one for sending probes and another one for waiting for probes while measuring the delay.
   (a) *Probing Function*: The probing function is designed to estimate the delay on overlay o_{ijk} for all k in O_{ij}. Controller C_i continuously sends probe packets at frequency f over all overlays k in O_{ij} towards each destination site s_j.
   (b) *Measurement Function*: Controller C_i waits for probe packets from all connected sites. Upon receiving a probe, it examines the sending site and the overlay used. It extracts the timestamp contained within and calculates the time difference with the current timestamp. The corresponding d_{ijk} value is then updated.
3. *Northbound API*: The northbound API of controller C_i implements a REST API to receive decisions (i.e. the selected $o_{ijk} \in O_{ij}$) from agent A_i. Once a decision is received, the configuration function that updates the current configuration accordingly is triggered. In response, the monitored delays d_{ijk} for all j and k are sent to the orchestrator.
4. *Southbound API*: The southbound API is implemented using OpenFlow. Controller C_i sends and receives OpenFlow messages to and from ER_i. Specifically, the southbound API is utilized to send probes traversing all overlays between s_i and s_j (i.e. $o_{ijk} \in O_{ij}$) through PacketOut messages. It is also used to update routes on the ER, in order to select a different overlay based on the decisions of A_i and to receive probe packets sent by other controllers.
5. *EastWest API*: EastWest APIs are necessary for inter-controller information exchange. Information is shared for exchanging monitoring data. Specifically, each controller C_i sends its measured d_{jik} values to C_j, and in return, it receives d_{ijk} values from C_j.

4.2. Orchestration plane

The orchestration plane is responsible for the analysis and decision phases of network policies, and it is based on RL. As demonstrated in [6], a single-agent approach is overly complex and leads to an exponential increase in the state and action space. Therefore, in this paper, we follow the Multi-Agent Reinforcement Learning with Networked Agents (MARL-NA) approach. This approach, while keeping the decisions entirely independent to ensure robustness and reduce solution complexity, utilizes a reward exchange mechanism that enables the implicit cooperation among agents, by taking into account the mutual effect of the combination of their actions (see Fig. 5).

4.2.1. Reinforcement learning solution

In this section, the RL-solution design is presented. RL involves two key components: the agent and the environment. The agent can execute a set of actions that cause changes in the environment's state along with the reception of rewards or penalties by the agent based on the efficacy of such actions [34]. By performing repeated trial and error, the agent acquires knowledge about the environment and comes up with the most successful strategy to accomplish a certain set of goals, all without any prior awareness of the environment.

Environment: There is a single environment that models the WAN and includes sites, overlays among sites, and ERs. In practical scenarios, complete visibility of the entire network is often unattainable. Specifically, we consider a typical SD-WAN scenario where overlay networks are not under the direct control of the enterprise operating the SD-WAN. As a result, we lack information about the internal details within the depicted "clouds" in Fig. 2 (such as internal topology, link capacity, and traffic traversing links). Our assumption is that only certain network parameters can be monitored from the network edges by means of active probes (i.e. the end-to-end delay).

Rewards: Unlike typical RL contexts, where rewards are directly provided by the environment (e.g., gaming), we designed specific reward functions in this scenario. Essentially, there is a reward (positive return) when the network policy is adhered to, or a penalty (negative return) when the policy is violated. For example, when the policy is entirely dependent on QoS, the reward is determined by the real-time monitored delay on the selected overlay with respect to the preset delay threshold that represents the maximum latency tolerated.

Actions: As mentioned earlier, overlay networks are frequently beyond the direct control of the enterprise. Consequently, we assume that TE policies cannot be applied within the overlay connections. While traditional methods assume that the core network is controllable and act upon that, for example, changing link weights and altering in this way the routing protocols, in our model, the only possible control action involves instructing ERs to select one of the available overlays for forwarding traffic to each destination site.

Within the several model-free methods, the most successful ones for WAN path selection are those that rely on the state–action-value function, specifically, Q-learning [11]. At each timestamp t, the agent selects an action a_t, and consequently enters a new state s_{t+1} and receives a reward r_t.

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a) - Q(s_t, a_t)) \quad (1)$$

The algorithm updates the Q-value $Q(s_t, a_t)$, which estimates the value of performing action a_t in the state s_t, using the Bellman equation

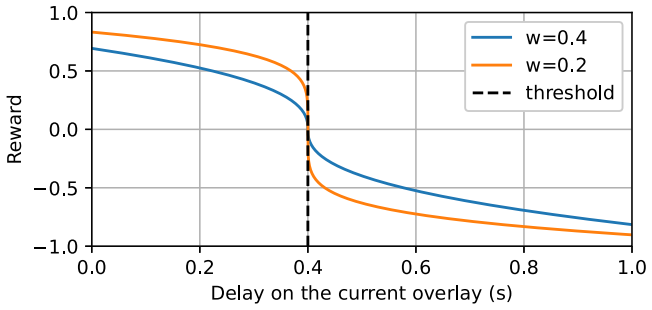


Fig. 6. Reward computation as a function of the experienced delay, the policy threshold, and weight w.

(Eq. (1)). The learning rate α determines the balance between current and past experiences, while the discount factor γ determines the relative importance of future rewards compared to the current reward. The immediate reward received from the environment is denoted as r_t. The exploration rate ϵ determines the extent to which the agent explores the state environment or exploits the learned knowledge. At state s_t, a random action is chosen with a probability of ϵ, while the action with the highest Q-value is chosen with a probability of $1-\epsilon$. Parameters α and γ are tuned through grid search, while ϵ is initially set to its maximum value ϵ_{max} to facilitate rapid learning by the agent. Subsequently, it decays to a minimum value ϵ_{min} to progressively shift towards the exploitation of the best actions. In particular, the *epsilon-greedy decay method* is employed, where the exploration rate decays at an exponential rate ϵ_{decay} as a function of the experiment step t, as specified by Eq. (2). As the experiment progresses – with the increase of t – the second term of the equation decreases, causing the exploration rate to shift towards its minimum value.

$$\epsilon_t = \epsilon_{\min} + (\epsilon_{\max} - \epsilon_{\min}) \cdot e^{-\epsilon_{\text{decay}} \cdot t} \tag{2}$$

The objective of Q-learning is to maximize the cumulative reward $\sum_{t=0}^{\infty} \gamma^t r_{t+1}$.

4.2.2. Q-learning model

The Q-learning scheme is defined by actions, states, and rewards. Actions are defined as $[\hat{O}_{ij}(t)]$, where $\hat{O}_{ij}(t) = o_{ijk} \in O_{ij}$ if overlay o_{ijk} is chosen at time t for routing traffic from s_i to s_j, and hence can assume N_{ij} different values. Given $d_{ijk}(t)$, the delay measured on the kth overlay between s_i and s_j at time t, and T_{ij}, the delay threshold set with the network policy for the communication from site s_i to s_j, states are defined by the tuple $[\hat{O}_{ij}(t-1), x_{ij}(t)\,]$, where $\hat{O}_{ij}(t-1)$ is the overlay used to route traffic between s_i and s_j at time t and is hence equal to the action chosen at time $t-1$, and $x_{ij}(t)$ is a state variable taking into account if the SLA is being violated or not (Eq. (3)). Specifically, if the delay on the currently chosen overlay to route traffic from s_i to s_j (i.e. $d_{\hat{O}_{ij}(t)}$) is less or equal to T_{ij} the state variable is equal to 0, otherwise it is equal to 1.

$$x_{ij}(t) = \begin{cases} 0 & \text{if } d_{\hat{O}_{ij}(t)}(t) \le T_{ij} \\ 1 & \text{otherwise} \end{cases} \tag{3}$$

Dynamic reward functions are designed. The reward R at time t is calculated as follows:

$$R(t) = -\frac{\left| d_{\hat{O}_{ij}(t)} - T_{ij} \right|^{w}}{d_{MAX}(t)} \cdot sign(d_{\hat{O}_{ij}(t)} - T_{ij}) \tag{4}$$

w is an exponent with real values between 0 and 1 that adjusts the sensitivity of the reward to the deviations from the threshold, and $d_{MAX}(t)$ indicates the largest delay encountered at time t and is used for normalization. A little deviation from the threshold will result in a low payout when w is high. Low w values, in contrast, highlight minor discrepancies. A positive reward is given for delay below the threshold, and a negative reward for delay over the threshold is the result of multiplying the obtained value by the opposite of the sign of $d_{\hat{O}_{ij}(t)} - T_{ij}$. The possible range for the final reward is between −1 and 1.

The potential benefits for various delay levels are shown in Fig. 6, with the threshold set at 400 ms and the maximum delay assumed to be $1s$. When the delay is less than the threshold, rewards are positive, and vice versa when the delay is more than the threshold. It is also noted that when all overlays fail to meet the QoS requirement, a negative reward is incurred irrespective of the overlay used. Nevertheless, the penalty in this case is determined by the absolute difference between the experienced delay and the set threshold. This implies that the greater the violation of the policy, the more substantial the penalty. For this reason, in this scenario, the action of choosing the overlay exceeding the threshold by the greatest margin is penalized more, encouraging the usage of the overlay violating the threshold by less.

The calculation of the reward for policy B differs slightly to take into account the different transit costs of the overlays. Although the absolute value of the reward remains consistent with that of policy A, the sign changes when multiple overlays encounter a delay beyond the threshold. In this scenario, if the overlay with the higher cost is employed, the resulting reward is negative. This aligns with the second condition for the violation of Policy B, as introduced in Section 3. Specifically, if $d_{\hat{O}_{ij}(t)} \le T_{ij}$ but $\exists o_{ijk} \in O_{ij} : o_{ijk} \neq \hat{O}_{ij}(t) \wedge d_{o_{ijk}(t)} \le T_{ij} \wedge cost(o_{ijk}) \le cost(\hat{O}_{ij}(t))$, the reward is negative.

The process of computing rewards must also consider the cost incurred for a network reconfiguration. Whenever the overlay used for routing traffic is changed, the service is impacted for a certain time. In order to discourage the agent from constantly changing the overlay, a minor detrimental penalty p is incorporated into the reward function each time the overlay is changed. The algorithm operates at a specific frequency f. At regular intervals of $\frac{1}{f}s$, the agent executes an action and is subsequently given a reward. It is important to note that the agent is rewarded even when $\hat{O}_{ij}(t) = \hat{O}_{ij}(t-1)$, meaning that it did not modify the configuration. This is done to account for potential changes in the environment state that are not influenced by the actions of the agent. In fact, we consider the environment to be highly variable due to uncontrollable factors and also due to the actions of other agents. The frequency f, should be as high as possible to quickly operate a reconfiguration when necessary. Nevertheless, the choice of f is constrained by the duration required for the system to identify an infraction of policy and carry out a reconfiguration.

4.2.3. MARL-NA model

Let $G = (A, E)$ be a directed graph where A is the set of agents and E represents the set of edges. Since each agent is responsible for a single SD-WAN site, $|A| = M$. $e_{ij} \in E$ means agents i and j can communicate with each other through this edge. Two agents i and j are considered neighbors and are therefore allowed to communicate if agent j can affect agent i in the active link set L, or vice versa. We define N_i the set of Neighboring Agents of A_i: A_j is in the neighboring set N_i if exists $e_{ij} \in E$ or $e_{ji} \in E$.

A networked multi-agent Markov Decision Process (MDP) is characterized by a tuple $(\mathcal{X}, \{\mathcal{A}_i\}_{i\in M}, \{R_i\}_{i\in M}, G, \lambda)$: $\mathcal{X}$ is the global state space shared by all agents (we consider the state to be partially observable, i.e. each agent observes its own state $\mathcal{X}_i$); $\mathcal{A}_i$ is the action space of agent i, while $\mathcal{A} = \prod_{i=1}^{M} \mathcal{A}_i$ is the joint action space; R_i denotes the local reward function of agent i. In fully decentralized MARL, the agents act in a completely independent way. Each agent just receives its own reward $R_i(s_t, a_t)$, and the environment switches to the new state s_{t+1}. In MARL-NA, each agent interacts with the other agents in its neighboring set N_i. Each agent A_i receives its local reward $R_i(s_t, a_t)$ and also $R_j(s_t, a_t)$ from the other agents A_j in the neighboring set N_i.

Algorithm 1 Agent i Control Loop

Require:
Run control plane (configuration, monitoring and APIs)
Select Policy P (A or B)
States $\mathcal{X}_i = \{1, \ldots, n_x\}$
Actions $\mathcal{A}_i = \{1, \ldots, n_a\}$, $\mathcal{A}_i : \mathcal{X}_i \Rightarrow \mathcal{A}_i$
Reward function $R : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$
Learning rate $\alpha \in [0, 1]$
Discounting factor $\gamma \in [0, 1]$
Exploration rate $\epsilon \in [0, 1]$
Agent weighting factor $\lambda \in [0, 1]$

procedure QLearning($\mathcal{X}_i$, $\mathcal{A}_i$, R, T, α, γ)
  Initialize $Q : \mathcal{X} \times \mathcal{A} \to \mathbb{R}$ arbitrarily
  while Q is not converged **do**
    Get g state $s \in \mathcal{X}_i$ ▷ Choose action a:
    if random.uniform$(0, 1) < \epsilon$ **then**
      $a \leftarrow$ random.sample($\mathcal{A}$)
    else
      $a \leftarrow \arg\max_a Q(s, a)$
    $r_i \leftarrow R(s, a)$ ▷ Receive and adjust the reward
    if $a_i \neq a_{old}$ **then**
      $r_i \leftarrow r_i - p$
    $r \leftarrow r_i \cdot (1 - \lambda) + \lambda \cdot \sum_j NR_{ij}$
    $s' \leftarrow env.getState()$ ▷ Receive the new state
    $Q(s', a) \leftarrow (1 - \alpha) \cdot Q(s, a) + \alpha \cdot (r + \gamma \cdot \max_{a'} Q(s', a'))$
    $s \leftarrow s'$
    $a_{old} \leftarrow a$
    Decrease ϵ

For simplicity, we define NR_i the set of rewards of the agents in N_i. In this scenario A_i, once received R_i, adjusts it also considering NR_i according to the following equation:

$$r = r_i \cdot (1 - \lambda) + \lambda \cdot \sum_j NR_{ij} \tag{5}$$

where λ is used to weigh the reward received by A_i with respect to the rewards obtained by the other agents, similarly to [35].

The objective is to maximize the team-average reward, denoted as $\bar{R} = \frac{1}{M} \sum_{i \in M} R_i$, where M represents the set of agents and R_i is the reward function of agent i.

Collaboration protocol among agents. In MARL-NA, agents are permitted to communicate only if they are considered neighbors. To mitigate complexity, it is crucial to minimize interactions between agents. Furthermore, considering irrelevant information in the computation of an agent's adjusted reward can introduce noise, potentially degrading the agent's performance. Therefore, in our scenario, we define that two agents A_i and A_j, are considered neighbors if the network sites they manage S_i and S_j are connected to a common overlay or an overlay with at least one common underlay link. This condition encompasses cases where the actions chosen by A_i may influence those chosen by A_j, and vice versa. This consideration arises from the fact that agents sharing at least one overlay or a link within the overlay are more likely to encounter conflicts. For instance, their actions might lead to selecting the same overlay for routing traffic to another site, potentially straining the overlay's bandwidth. It is also worth noting that the reward adjustment follows Eq. (5), only when the action chosen by A_i corresponds to the selection of the shared overlay. To address this, we assume that we possess knowledge about the overlays shared between agents. This information can be obtained through correlation analysis, as demonstrated in [7], or it may be transparently shared by the entity overseeing the possibly shared overlays (e.g. the telco provider).

In Algorithm 1, the single agent Q-Learning procedure is detailed. The algorithm begins by initializing various parameters, including the state space $\mathcal{X}$, action space $\mathcal{A}$, reward function R, threshold T, learning rate α, discounting factor γ, exploration rate ϵ, and agent weighting factor λ. The core of the algorithm is the Q-Learning loop. Within the loop, the agent selects an action a (i.e. the overlay to use) based on an exploration–exploitation strategy: it either explores a random action with a probability of ϵ or exploits the action with the maximum Q-value. The agent receives a reward r_i based on the chosen action a in the current state s by the environment. If the action a_i is different from the previous action a_{old}, a penalty p is subtracted from the reward r_i. In case the action selected a_i corresponds to choosing an overlay shared with other agents, the reward is then adjusted using a weighted combination of the local reward received by the environment and the sum of rewards from other agents (NR_{ij}). The agent transitions to a new state s' in the environment, and the Q-value for the selected action is updated based on the observed reward and the maximum Q-value in the new state. As time progresses, the exploration rate ϵ is systematically reduced to increasingly shift the agent's focus from exploring the environment towards exploiting its existing knowledge.

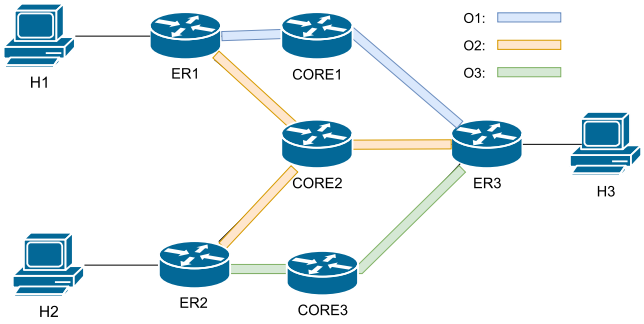


Fig. 7. Topology Under Test: 3 Sites connected by means of 3 Overlays (O_1, O_2, and O_3).

5. Experimental evaluation

In this section, we aim to thoroughly assess the effectiveness of our framework in managing network policies, optimizing QoS, and reducing costs, with a direct comparison to baselines.

The experimental topology and the network setup are described in Section 5.1. In the first set of tests, our framework is evaluated for Policy A considering two scenarios, one in which traffic flows from different sites have different QoS requirements and one in which the QoS requirements are the same. Subsequently, we perform experiments by applying two different network policies (i.e. Policy A and Policy B) to traffic flows from different sites to evaluate our framework in the case of both QoS and cost constraints. For each scenario, we perform parameter tuning and then evaluate the results by considering metrics presented in Section 5.2.

5.1. Experimental setup

The experiments are conducted within an Ubuntu 18.04 x64 Virtual Machine environment, hosted on a KVM hypervisor. The virtual machine is equipped with 16 AMD EPYC-Rome Processors, each with 64 cores running at 3.35 GHz, and has 64 GB of RAM allocated for the experiments. The experimentation setup utilizes the Mininet 2.3.0 network emulator, with Python 3.10.12 installed. For network control, the Ryu 4.34 controller is employed. Additionally, the OpenAI Gym library version 0.26.2 is used to create the RL environment.

Topology under test. The SD-WAN configuration considered is depicted in Fig. 7. The topology is emulated using the Mininet network emulator. Each remote site is represented by a host, responsible for receiving or transmitting SD-WAN traffic, and an Edge Router. The ERs

Table 2
Parameters for each Scenario. Scenario I: Weighting For Fairness; Scenario II: The Tragedy of The Commons; Scenario III: Heterogeneous Policies (D: Nominal Delay Value, DVar: Maximum Delay Variation).

Scenario	Policy A_1	Policy A_2	T_{13}	T_{23}	D (O_1)	DVar (O_1)	D (O_2)	DVar (O_2)	D (O_3)	DVar (O_3)
I	A	A	400 ms	200 ms	200 ms	700 ms	100 ms	700 ms	100 ms	700 ms
II	A	A	300 ms	300 ms	150 ms	700 ms	50 ms	700 ms	150 ms	700 ms
III	A	B	250 ms	250 ms	100 ms	700 ms	100 ms	700 ms	50 ms	0 ms

Table 3
Parameters chosen for MARL-NA and baselines.

Parameter	Value
Discount Factor (γ)	0.1
Exploration Rate (ϵ)	$\epsilon_{min} = 0.001$ $\epsilon_{max} = 0.8$ $\epsilon_{decay} = 0.02$
Learning Rate (α)	0.3
Weight (w)	0.1
Frequency (f)	$(1/0.5)\ s^{-1}$
Penalty (p)	0.3
Constant (k)	0.2

are implemented as OpenvSwitches and are configurable by the Ryu controllers to dynamically change the overlay used for specific destinations. These ERs also play a crucial role in monitoring, as detailed in Section 4.1. The various overlays are emulated using links whose specific nominal delay and capacity values are assigned, and basic non-controllable core switches are used to emulate queuing delays. To prevent bottlenecks at edge sites, the external links – corresponding to WAN connections – are set to a capacity of 6 Mbps. Conversely, the internal links, those connected to end devices, are set to a capacity of 3 Mbps. To emulate the variability of Internet network conditions, *netem* is used to set the delay of the internal WAN interfaces to be at a certain value ±a constant expressed in milliseconds with the next random delay value correlated with the last one, with the correlation expressed as a percentage.

Orchestrators and Controllers. Every site within the experiment is managed by an individual instance of the Ryu controller, which implements the network functions as described in Section 4.1. The Ryu controller instances communicate with one another through REST APIs in order to exchange information. Additionally, each controller instance features an API server that can be queried by the orchestration plane to request configuration changes and receive updated delay measurements. The orchestration plane itself is composed of Network Agents that implement the algorithm detailed in Section 4.2, along with an RL Environment modelled using the OpenAI Gym library. This environment abstracts the underlying network, querying the parameters for the computation of network states and rewards from the controllers.

Fig. 7 shows the experimental setup in Mininet. It represents a classical hub-and-spokes use case. In a hub-and-spokes SD-WAN scenario, two remote sites route their network traffic through a central hub location, such as a headquarters or a cloud-based data center. H_1 and H_2 represent hosts located within remote enterprise sites, while H_3 represents a host located in headquarters or in the cloud. H_1 and H_2 can communicate with H_3 by means of 3 different overlays (namely O_1, O_2, O_3). Specifically, H_1 can use O_1 or O_2 to communicate with H_3, while H_2 can use O_2 or O_3 to communicate with H_3. F_1 and F_2 are two constant bitrate (1.6 Mbps) traffic flows generated from H_1 to H_2 and from H_1 to H_3 respectively; F_1 is controlled by A_1 while F_2 is controlled by A_2. The communication paths are bidirectional. Static routes are set on the intermediate switches for reachability. Intermediate switches are not configurable since they represent the non-controllable underlay network. Conversely, S1, S2, and S3 are controllable Openflow switches, that can be configured by the controller to choose a particular overlay for sending traffic to the destination site. Table 2 summarizes the policy established for each agent (Policy A or Policy B), the QoS requirement in terms of the maximum allowed latency (T_{13} for A_1 and T_{23} for A_2), and eventually the delay set on each overlay in terms of nominal value and possible deviation from it for each different experiment.

5.2. Evaluation metrics

The proposed framework is evaluated based on a set of metrics intended to assess its capability of meeting the configured policies. Specifically, we evaluate the SLA violation ratio (SVR). SLA is considered violated if the maximum latency expressed with the configured network policy is exceeded. SVR is defined in Eq. (6) as the ratio of SLA violations for each policy over the total number of policy checks conducted, in which the Policy is checked at frequency f.

$$SVR = \frac{\text{SLA Violations}}{\text{Total SLA Checks}} \tag{6}$$

We also evaluate the Policy Violation Ratio (PVR) both for Policy A and Policy B. The policy is considered violated according to the conditions explicated in Section 3. PVR is defined as in Eq. (7) as the ratio of policy violations for each policy over the total number of policy checks conducted, in which the Policy is checked at frequency f.

$$PVR = \frac{\text{Policy Violations}}{\text{Total Policy Checks}} \tag{7}$$

While SVR is a good indicator of QoS assurance, PVR provides information on how the system adheres to the policy. For example, if Policy A is chosen and all the overlays are experiencing a delay that surpasses the threshold, SVR is impacted because the user experiences poor network performance, whereas PVR is not affected because the algorithm could not make a better choice. Regarding Policy B, while SVR only gives an indication of User Experience, PVR also takes into account the overlay transit cost.

To also assess the impact of violations on QoS directly, we evaluate Latency, measured at frequency f. Note that in the context of orchestration decision-making, latency is measured through the control plane monitoring function. On the other hand, for evaluation purposes, we consider the end-to-end delay measured between hosts located at the WAN edges. These measurements are computed based on logs produced by the D-ITG traffic generation tool [36].

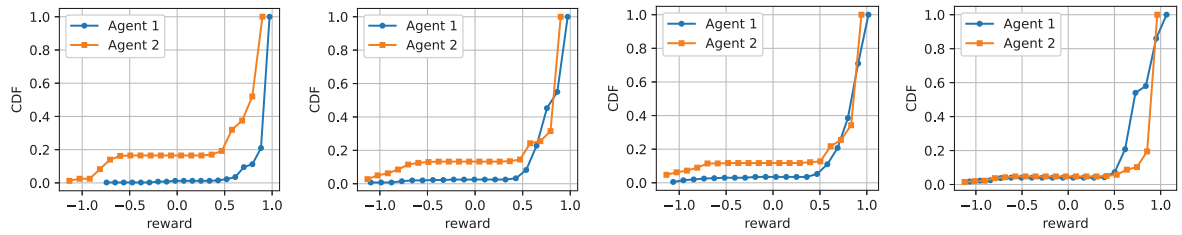
For RL-based systems, another significant evaluation metric is the current reward achieved by an agent from the environment. Also, this metric will be evaluated in experiments involving MARL-NA.

5.3. Baselines

The proposed framework is assessed by comparing it to two baseline approaches. These baselines serve as reference points for assessing the effectiveness and performance of our framework (**MARL-NA**).

Baseline 1: The first baseline consists of a Blind Algorithm (**BA**) that operates on the principle of blind switching. When a policy violation is detected, the traffic is blindly rerouted to a different randomly-chosen overlay network. This works only by knowing the delay on the current overlay as it happens in a passive monitoring fashion.

Baseline 2: A Greedy Algorithm (**GA**) that at each timestamp chooses the overlay that is experiencing the lowest delay by less than a constant k. This constant is set to prevent the so-called "channel flipping problem". This exploits the active probing mechanism to make informed decisions on the overlay to choose.



(a) $\lambda_1 = 0, \lambda_2 = 1$ (b) $\lambda_1 = 0.5, \lambda_2 = 0.5$ (c) $\lambda_1 = 0.8, \lambda_2 = 0.2$ (d) $\lambda_1 = 1, \lambda_2 = 0$

Fig. 8. Scenario I: Tuning the weight of the different agents rewards for cooperation. By increasing λ_1 and decreasing λ_2, A_1 manages to get higher rewards.

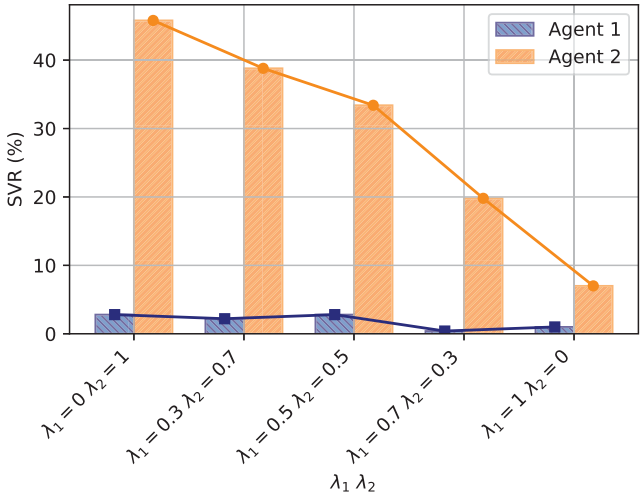
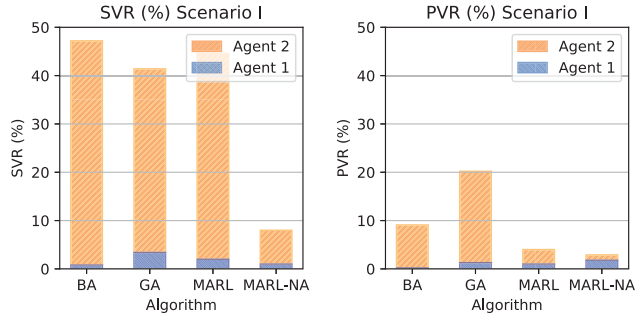


Fig. 9. Scenario I: Tuning the weight of the different agents' rewards: SVR as a function of λ_1 and λ_2 for F_1 and F_2.



(a) *SVR* incurred by A_1 and A_2. (b) *PVR* incurred by A_1 and A_2.

Fig. 10. Scenario I: SVR and PVR incurred by MARL-NA approach ($\lambda_1 = 1, \lambda_2 = 0$) compared to baseline. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Baseline 3: The third baseline involves a fully decentralized Multi-Agent Reinforcement Learning (**MARL**) approach, similar to our proposed framework, but with a key difference. In this baseline, each network agent operates independently without communication or coordination with neighboring agents. It considers only the local reward signal as there is no reward exchange among distinct agents. By comparing our framework to this decentralized MARL baseline, we can assess the added value of networked agents and cooperative decision-making in our approach.

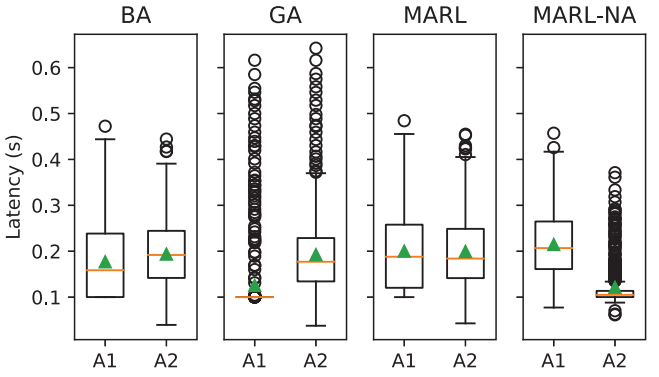


Fig. 11. Scenario I: Latency obtained using MARL-NA approach ($\lambda_1 = 1, \lambda_2 = 0$) by A_1 and A_2 compared to baselines.

The parameters chosen for the MARL-NA approach and the baselines are reported in Table 3.

5.4. Results

5.4.1. Scenario I: Weighting for fairness

In this experiment, A_1 and A_2 have to meet policies concerning two distinct Quality of Service (QoS) requirements. The first agent has a more relaxed requirement, namely "*Traffic must experience a maximum delay of 400 ms*", while the second agent has a more stringent requirement "*Traffic must experience a maximum delay of 200 ms*". Both sites generate traffic in a manner that, when both agents choose the common overlay (i.e. O_2), they saturate it, causing congestion and an increase in the delay. Consequently, the two objectives are in conflict, leading to competition between the two agents for the common resource. In fact, the common overlay (i.e. O_2) offers a lower nominal latency compared to O_1, and A_2 is willing to route traffic by using O_2 in the time intervals in which O_3 does not guarantee to meet the SLA. As a result, in a fully decentralized setting, A_1 would prefer O_2, even though O_1 would typically allow it to satisfy its requirement for most of the time to reach its objective. On the other hand, A_2, with its strict requirements, would not be able to meet them using O_3 exclusively and would benefit from the common overlay O_2. However, it is in competition with A_1 for accessing this resource.

In an entirely decentralized approach (MARL baseline), each agent would strive to achieve its own objective, even at the expense of other agents. For instance, A_1, which is rewarded for attaining the lowest possible latency relative to the threshold, would seek to consistently utilize the shared overlay, as it guarantees higher rewards. Consequently, the second agent is compelled to use O_3. Even though O_3 has the same characteristics as O_2, its variability causes the delay to surpass the threshold at some times, but in these cases shifting traffic to O_2 would cause the delay to increase immediately since A_1 is choosing O_2 as well, saturating the bottleneck link.

On the contrary, with the MARL-NA approach, A_1 and A_2 are considered neighbors because they share an overlay. For this reason, they exchange their local rewards. As described in the algorithm in Section 4.2.3, each agent adjusts its own reward, taking into account the reward of the other agent when the overlay used at time t is shared or has some link shared with another agent. For this reason, when A_1 greedily chooses O_2 and consequently A_2 experiences bad performances, A_1 also receives a negative reward, which allows it to adjust its behavior, favoring cooperation.

In Fig. 8 we present the results of the tuning of the parameter λ using our proposed MARL-NA approach. λ_i represents the weight assigned to the rewards of other agents, while (1-λ_i) represents the weight given to one's own reward. For instance, $\lambda_1 = 0$ implies that Agent A_1 assigns zero importance to the rewards of other agents and full importance to its own. Given the presence of two competing agents in the described setup, we set the values as follows ($\lambda_2 = 1 - \lambda_1$) to prevent competition. In particular, choosing $\lambda_1 = 0$ implies $\lambda_2 = 1$, meaning that A_2 will give exclusive relevance to A_1's reward.

Each subplot in Fig. 8 displays the Cumulative Distribution Function (CDF) of the rewards achieved by A_1 and A_2 under specific values of λ. Initially, when $\lambda_1 = 0$ and $\lambda_2 = 1$, both A_1 and A_2 place sole importance on the reward of A_1. In this setup, A_2 receives 20% negative rewards, while A_1 predominantly receives positive rewards. As we increase the value of λ_1, consequently reducing λ_2, more significance is attributed to A_2's rewards compared to A_1's. This is reflected in the fact that the CDF of A_1 progressively shifts downwards, encompassing an increasing number of rewards greater than zero. Conversely, the CDF of A_2 slightly shifts left, including lower rewards, as it can be especially noticed in the last CDF. Interestingly, increasing the weight of A_2's reward does not significantly reduce A_1's reward. This can be explained by the fact that A_1 has a less stringent objective compared to A_2 (with thresholds $T_1 = 400$ and $T_2 = 200$): A_1 manages to achieve its objective even when assigning less weight to its own reward.

Fig. 9 shows SVR for A_1 and A_2 varying λ_1 and λ_2. By increasing λ_1 and decreasing λ_2, more importance is given to A_2 local rewards, and less importance is given to A_1 local rewards. Consequently, SVR_{A1} decreases, reaching the minimum value for $\lambda_1 = 1$ and $\lambda_2 = 0$. At the same time SVR_{A1} keeps to small values even if A_1 is penalized. Despite the fact that A_1 gets the lowest SVR for $\lambda_1 = 0.7$ and $\lambda_2 = 0.3$, the minimum overall SVR (obtained by considering both SVR_A and SVR_B) is achieved for $\lambda_1 = 1$ and $\lambda_2 = 0$. In this situation, r_2 (i.e. the local reward of agent A_2) assumes significantly greater importance in the overlay choice compared to r_1, confirming that weighing the agent with the stricter requirement improves the overall network performance and the capability to achieve global SD-WAN policies.

Fig. 11 presents the boxplots of latency for the baselines and for the MARL-NA approach. The MARL-NA approach stands out as the only one capable of achieving latency below the threshold ($T = 200$ ms) for A_2, notwithstanding the presence of outliers. It also manages to maintain latency below the threshold for A_1 (in this case, the threshold is set at $T = 400$ ms). GA is the sole approach that exhibits significantly better performance with respect to the MARL-NA approach, for what concerns A_1. However, this greedy behavior has a notable adverse impact on the latency experienced by A_2.

Summing up, we adopt the *Weighting for Fairness* method: if λ was set to 0.5 for both agents, it would imply an equal weighting of their objectives, which may not be appropriate because A_1's requirements are less strict. Setting λ to 0.7 for A_1 and 0.3 to A_2 acknowledges the asymmetry in the requirements and ensures that A_2's policy has a more substantial influence on the overlay choice.

Fig. 10 displays the SLA and the Policy violation percentages incurred by the MARL-NA approach compared to the baselines. The SVR of A_1 is represented in blue, while the SVR of A_2 is in orange. As shown in Fig. 10(a), in the fully decentralized MARL approach, a very high percentage change in SLA violations is observed, even higher than GA and similar to BA. In contrast, thanks to cooperation reached among agents, the proposed MARL-NA approach succeeds in significantly reducing SLA violations, reaching an overall SVR under 10%. In Fig. 10(b) we also notice that MARL-NA manages to achieve a small number of policy violations PVR compared to the baseline approaches. Furthermore, it is evident that MARL-NA reduces the percentage of SLA violations, especially for A_2. This reduction is attributed to the selection of the parameter λ, which places greater weight on A_2's reward compared to A_1's. Nevertheless, even A_1's SVR remains low, thanks to cooperation that keeps latency low on the shared overlay, which also favors A_1.

Table 4
Scenario I: Percentage of use of the common overlay for each agent varying the reward's weights.

λ_1, λ_2	O_2 usage by A_1 (%)	O_2 usage by A_2 (%)
0.6, 0.4	62.5%	61.3%
0.7, 0.3	64.8%	59.8%
0.8, 0.2	45.8%	65.7%
1, 0	36.0%	79.3%

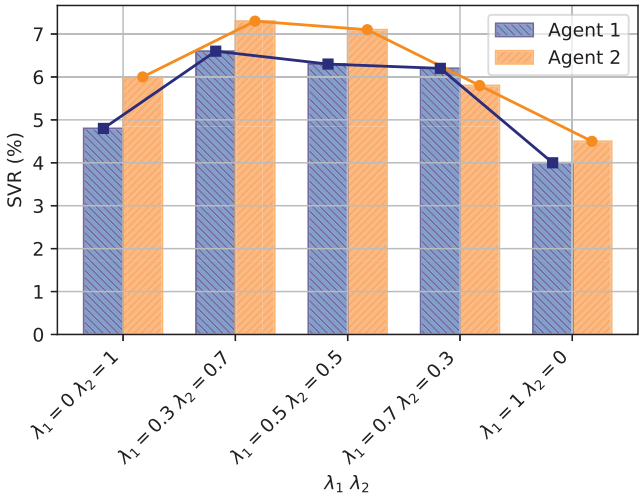


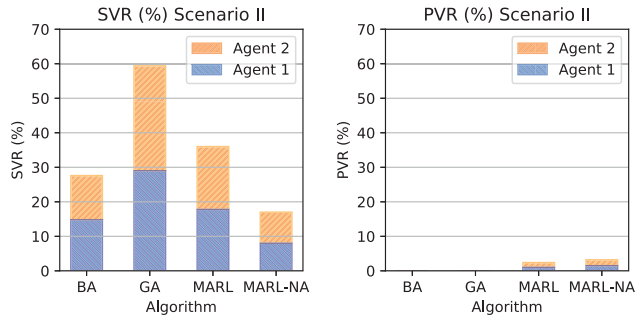
Fig. 12. Scenario II: Tuning the weight of the different agent's rewards: SVR of F_1 and F_2 for different values of λ.

In fact, the cooperative algorithm prevents the use of the shared path O_2 by A_2 since it has a lower latency requirement, and it almost always suffices with its overlay O_1 to avoid SLA violations. This is shown in Table 4: this table displays the percentage of the utilization of the common overlay O_2 by flows F_1 and F_2 controlled by A_1 and A_2 respectively, for increasing values of λ_1 (g from $\lambda_1 = 0.6$) that progressively favor the reward of A_2 over A_1. As it can be observed, the usage of O_2 by A_1 gradually decreases, while the usage of O_2 by A_2 progressively increases.

5.4.2. Scenario II: The tragedy of the commons

We set a scenario where all exclusive overlays of agents exhibit similar characteristics, while the common overlay boasts superior features (with a nominal latency of 50 ms compared to 150 ms with a deviation of 700 ms in other overlays). Furthermore, the traffic from s_1 and s_2 is insufficient to saturate the exclusive overlays individually. On the contrary, the combination of traffic from both sites can lead to the saturation of the common overlay, potentially causing uncontrolled congestion and eventually resulting in increased delays. In this experimental setup, we investigate the MARL-NA approach with agents having the same objective: "*Traffic must experience a maximum delay of 300 ms*".

As in the previous scenario, λ-tuning is performed. Fig. 12 shows the SVR obtained by A_1 and A_2 varying λ_1 and λ_2. In this scenario, SVR



(a) SVR incurred by A_1 and A_2. (b) PVR incurred by A_1 and A_2.

Fig. 13. Scenario II: SVR and PVR incurred by MARL-NA approach ($\lambda_1 = 0, \lambda_2 = 1$) compared to baseline.

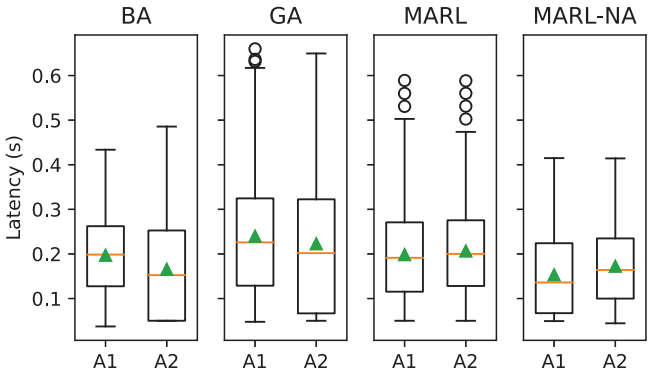


Fig. 14. Scenario II: Latency obtained using MARL-NA approach ($\lambda_1 = 0, \lambda_2 = 1$) by A_1 and A_2 compared to baselines.

is less sensitive to the values of λ_1 and λ_2 with respect to the previous case, as long as the $\lambda_1 = 1 - \lambda_2$ convention is kept. This is explained by the fact that in this scenario agents have the same requirement in terms of delay and the overlays' conditions are similar. Surprisingly, it can also be observed that agents exhibit better coordination when both prioritize either r_i or r_j, meaning when either $\lambda_1 = 0$ and $\lambda_2 = 1$ or $\lambda_1 = 1$ and $\lambda_2 = 0$, which can be explained by the symmetry of the requirements as well. Let us suppose that $\lambda_1 = 1$ and $\lambda_2 = 0$. When A_1 is utilizing the shared overlay and receives a negative reward from A_2 due to an increase in latency on path O_3, it shifts traffic to O_1. This action opens up the opportunity for A_2 to access the shared overlay O_2 without increasing the delay on the shared overlay due to congestion. The opposite case occurs when we solely emphasize A_1's reward. When A_1 experiences negative rewards and A_2 is using O_2, A_2 receives the negative rewards from A_1 and shifts to O_3, ensuring A_1's access to the shared resource O_2, without exhausting O_2's bandwidth. It is important to underline that even the agent that is giving sole importance to the other's reward when using the shared overlay is benefitting from this mechanism since an increase in the delay of the common path also prevents them from meeting the SLA. This coordination is achieved in a weaker way when intermediate weights are assigned to the rewards of A_1 and A_2.

The configuration that achieves the smallest SVR is selected, and the corresponding SVR and PVR are then compared with the results of the baselines in Fig. 13. As shown in Fig. 13(a), the completely decentralized approach without coordination (i.e. MARL) obtains worse performance with respect to the BA baseline, while the BA algorithm manages to obtain lower SVR with respect to GA, since it acts less greedily, depleting the shared overlay less. Furthermore, the lack of cooperation when using MARL causes competition for the shared resource between the two agents, increasing the delay for both of them. Similarly to the "Tragedy of the Commons" economic theory, shared resources, when open to unrestricted use by individuals, tend to be overexploited and depleted, leading to negative outcomes for everyone involved. On the contrary, MARL-NA manages to avoid the "Tragedy", achieving better results compared to all the baselines in terms of SVR and also in terms of delay, as shown in Fig. 14, thanks to cooperation. To sum up, the cooperative approach not only helps in case of asymmetric conditions (e.g. different objectives as in Scenario I), but also in case of agents having the same objective, preventing the agents from exhausting the common overlay. Fig. 13(b) shows that, while BA and GA always respect the policy in this scenario, MARL and MARL-NA show a small number of policy violations, although this does not impact the SVR: quite the opposite, MARL-NA manages to meet SLA despite a small number of violations.

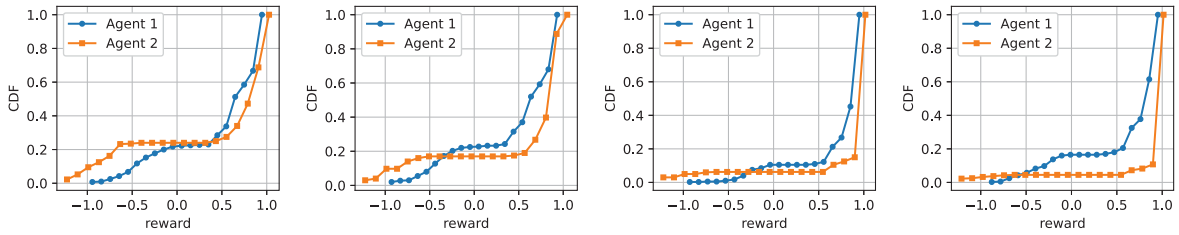
Let us assume that one agent decides to behave selfishly, disregarding the convention ($\lambda_i = 1 - \lambda_j$) to determine its reward. Instead, it chooses to use only its own reward – by setting $\lambda_1 = 0$ – without regard for the values established by other agents. While opting for a selfish choice may seem advantageous for the self-centered agent, it does not result in improved outcomes. In fact, in such scenario, we measured the SVR obtained in the classical cooperative approach with $\lambda_1 = 1 - \lambda_2$ and the one with $\lambda_1 = 0$ and we observed that the SVR increases by 5% for F_1 and by 2% for F_2. The increase is observed not only for F_2 – i.e. the flow controlled by the non-selfish agent – but also for F_1. The absence of gain for the selfish agent and, in fact, a deterioration in its performance can be attributed to the fact that cooperative behavior in a multi-agent system generally results in superior outcomes for all agents within the system. When an agent refuses to cooperate, it sabotages its own long-term success, since it exacerbates the competition for the shared resource (i.e. the shared overlay). Taking all of this into account, the absence of benefits for agents engaging in selfish behavior should naturally deter such actions, as the outcome not only deteriorates for other agents within the system but also fails to improve for the selfish agent.

5.4.3. Scenario III: Heterogeneous policies

In this experiment, we examine the behavior of two agents, each configured with a distinct Policy. Agent A_1 is configured with Policy A, whereas Agent A_2 is configured with Policy B. Policy B proves advantageous when dealing with overlays billed differently. In this scenario, O_3 represents an MPLS path that is billed with an additional cost proportional to the amount of traffic traversing the overlay. O_2, on the other hand, serves as a public Internet overlay lacking guaranteed QoS but offering a fixed cost, with no additional billing based on the bandwidth used. While the path based on the public Internet presents variable conditions, O_3 is a dedicated path that consistently ensures that the latency remains below the set threshold T_{23}. The reward structure of Policy B is designed to deter A_2 from utilizing the costly path (i.e. O_3). It does so by applying negative rewards when both the available overlays keep the delay below the threshold and A_2 opts for the costlier path (i.e. O_3).

In this experimental setup, we investigate the MARL-NA approach with agents having the same SLA objective: "*Traffic must experience a maximum delay of 250 ms*".

As in the previous scenarios, we perform λ-tuning in order to select the best configuration in terms of overall SVR. In Fig. 15 we observe that, g from $\lambda_1 = 0.3$ and $\lambda_2 = 0.7$ and increasing the importance given to r_2 with respect to the one given to r_1, agents are progressively receiving higher rewards, although for $\lambda_1 = 1$ and $\lambda_2 = 0$ the CDF of A_1 includes lower rewards. In a specular way, in Fig. 16 we observe that, by increasing λ_1 and decreasing λ_2, SVR_{A1} and SVR_{A2} increase up to a certain value and then start decreasing after $\lambda_1 = 0.5$ and $\lambda_2 = 0.5$. SVR_{A1} reaches its minimum value for $\lambda_1 = 0.7$ and $\lambda_2 = 0.3$ while SVR_{A2} reaches its minimum value for $\lambda_1 = 1$ and $\lambda_2 = 0$. A_1's SVR improves up to a certain level since it avoids using a potentially congested path. Nevertheless, A_1 can never achieve a SVR as small as the one of A_2 since it has no quality-guaranteed overlay available. It is



(a) $\lambda_1 = 0.3, \lambda_2 = 0.7$ (b) $\lambda_1 = 0.5, \lambda_2 = 0.5$ (c) $\lambda_1 = 0.7, \lambda_2 = 0.3$ (d) $\lambda_1 = 1, \lambda_2 = 0$

Fig. 15. Scenario III: Tuning the weight of the different agents rewards for cooperation.

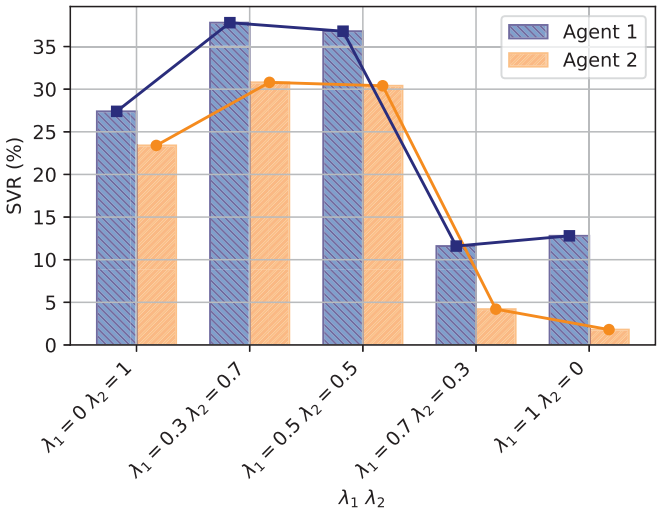
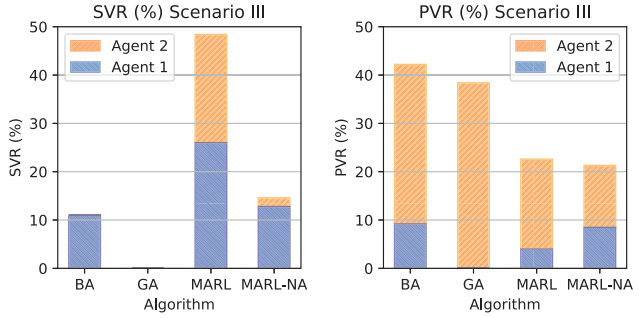


Fig. 16. Scenario III: Tuning the weight of the different agent's rewards: SVR of F_1 and F_2 for different values of λ.



(a) SVR incurred by A_1 and A_2. (b) PVR incurred by A_1 and A_2.

Fig. 17. Scenario III: SVR and PVR incurred using MARL-NA approach ($\lambda_1 = 1$, $\lambda_2 = 0$) compared to baselines.

also noticed that when switching from $\lambda_1 = 0.7$ and $\lambda_2 = 0.3$ to $\lambda_1 = 1$ and $\lambda_2 = 0$, the performance of A_2 improves in terms of SVR, while the performance of A_1 worsens. This is because, by increasing λ_1 and decreasing λ_2, more importance is given to A_2's reward.

As shown in Table 5, as we progressively increase the influence of A_2's local reward with respect to that of A_1, we observe a decrease in A_1's utilization of the common overlay, coupled with an increase in A_2's utilization of the same overlay. As A_1 relinquishes its hold on the common overlay, it provides A_2 with the opportunity to use it. Consequently, A_2 can begin to avoid utilizing the expensive dedicated path O_3, while at the same time keeping latency low.

We select the best configuration (i.e. $\lambda_1 = 1$ and $\lambda_2 = 0$) which minimizes the overall policy violations and compare it to the baseline approaches in Fig. 17. As depicted in Fig. 17(a), the MARL-NA approach

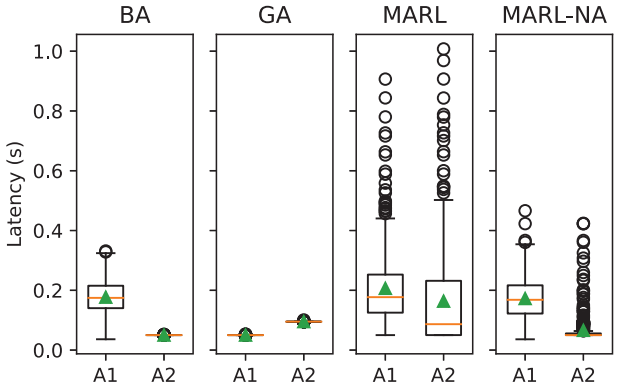


Fig. 18. Scenario III: Latency obtained using MARL-NA approach ($\lambda_1 = 1$, $\lambda_2 = 0$) by A_1 and A_2 compared to baselines.

Table 5
Scenario III: Percentage of use of the common overlay for each agent varying the reward's weights.

λ_1, λ_2	O_2 usage by A_1 (%)	O_2 usage by A_2 (%)
0, 1	67.2%	73.6%
0.3, 0.7	64.0%	73.6%
0.5, 0.5	64.6%	70.8%
0.7, 0.3	10.0%	93.25%
1, 0	10.4%	96.0%

notably achieves a substantially smaller number of SLA violations compared to the MARL approach, but it is also worth noting that both GA and BA achieve a null or low number of SLA violations. This trend is mirrored in latency as well, as it can be observed in Fig. 18. Latency experienced by F_1 and F_2 is kept very low by BA and GA, while MARL-NA notably outperforms MARL in keeping latency for both F1 and F2 under $T_{13} = T_{23} = 250$ ms.

However, GA and BA manage to achieve very low latency without taking into account the additional cost associated with O_3. In fact, the utilization percentage of O_3 by A_2 remains at 100% for both GA and BA, whereas it is kept very low for MARL and MARL-NA (respectively 6.75% and 4%). This means that MARL and MARL-NA are very advantageous in terms of economic cost. This is particularly evidenced in Fig. 17(b): while SVR does not evaluate the effectiveness of the approach in avoiding the costly path, PVR is impacted when using the costly path while another one which also meets the SLA is available. PVR incurred by A_2 results in fact very high for BA and GA baselines with respect to MARL and MARL-NA.

To sum up, MARL-NA is the only approach that achieves a low SLA violation rate consistently for both flows while simultaneously keeping the additional cost incurred by utilizing the QoS-guaranteed path at a minimum.

5.4.4. Discussion

In this section, we summarize the different experiments and scenarios tested with the proposed framework and present the main results

and takeaways. General considerations include the necessity of λ tuning to improve the performance of the cooperative MARL approach. However, the main result that emerged is that the MARL-NA approach, by exploiting cooperation among agents and preventing greedy behaviors, enhances performance compared to all the baselines and for the different scenarios taken into account.

In the first experiment (i.e. *Weighting For Fairness*), a scenario in which policies have varying requirements in terms of latency, along with asymmetric overlay conditions – i.e., different overlays' links have diverse latency requirements – is tested. Results show that by increasing the influence of the reward obtained by the agent with the strictest latency requirement, the overall SVR is significantly reduced compared to the greedy and totally decentralized algorithms.

In the second experiment (i.e., *The Tragedy of the Commons*), both agents have the same requirement in terms of the maximum allowed latency. The overlay shared between the two traffic types is more advantageous compared to the disjoint overlays, showing a smaller nominal latency. The cooperative MARL approach achieves fewer overall SLA violations compared to the greedy approach and the decentralized setting, in which both agents are willing to use the more convenient path, depleting, in this way, the bandwidth of the shared overlay. It is also observed that when the agent chooses to behave selfishly, disregarding the presence of other tenants, it does not achieve improved SVR. Refusing to cooperate, the agent sabotages its own long-term success.

In the third experiment (i.e *Heterogeneous Policies*), diverse types of policies are experimented with. Specifically, one agent is configured with a policy based on QoS, and the other one is based on both cost and QoS. In this case, favoring the reward of A_2 significantly improves the performance of both. The cooperative approach also improves performance in the case of heterogeneous policies. The more A_1 relinquishes its hold on the shared overlay, the more A_2 can avoid using the costly path. This altruistic behavior is absent in the case of the baselines.

Future Directions. In this paper, the sensitivity of the MARL-NA approach to the value of lambda, i.e., the importance given to individual agents' rewards, has emerged, especially in scenarios of asymmetric requirements, different overlay conditions, and in the case of heterogeneous policies (i.e., Scenarios 1 and 3). While for SD-WANs involving a few sites, it is manageable to explore all possible combinations manually, in large networks with numerous sites and non-disjoint overlays, proceeding in this way may be unfeasible. A more general approach, considering the automation of the λ choice, may be explored in future works. For example, this could involve performing a hyperparameter search to find the value that minimizes the average SVR or the maximum SVR. This search may be conducted using an optimization algorithm, such as one based on the gradient descent method.

6. Conclusions

Existing SD-WAN solutions fall short in delivering scalable and effective mechanisms to enforce global network policies, ultimately failing to meet the demanding requirements of QoS and cost efficiency critical for modern businesses.

In response to this pressing challenge, we introduced a decentralized control plane for SD-WAN, consisting of edge-based network monitoring for SD-WAN overlays and edge-driven configuration. Additionally, we devised a logically distributed orchestration plane comprising multiple RL agents, each of them making independent decisions for each SD-WAN site. This strategic design choice minimizes complexity while keeping the action and state spaces manageable, facilitating seamless scalability as new corporate sites are built. Moreover, we engineered a lightweight information-sharing mechanism among these agents, primarily driven by reward exchange. This mechanism fosters implicit cooperation without inflating the individual agent's complexity. Furthermore, our message exchange protocol between agents imposes minimal strain on the underlying architecture, consuming negligible bandwidth and engaging only neighboring agents in communication.

Our comprehensive evaluations within an emulated environment showed that the MARL-NA approach effectively upholds both QoS and cost efficiency while exhibiting fewer SLA violations compared to baseline methods in different scenarios. Furthermore, by meticulously tuning reward weights across agents, the MARL-NA approach enabled the prioritization of traffic with stringent requirements and facilitated the realization of global network policies. Lastly, it achieves a commendable reduction in transit costs when different billing schemes are used.

To sum up, our contribution encompassed a control and orchestration plane tailored for SD-WAN based on a cooperative version of Multi-Agent Reinforcement Learning, able to accommodate diverse network policies with varying QoS and cost objectives. This solution excels in scalability and robustness, and imposes minimal overhead on the underlying architecture. Furthermore, grounded in Reinforcement Learning, it demonstrates adaptability to fluctuating WAN conditions.

CRediT authorship contribution statement

Alessio Botta: Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **Roberto Canonico:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing. **Annalisa Navarro:** Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Giovanni Stanco:** Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **Giorgio Ventre:** Conceptualization, Methodology, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This work was partially supported by Cisco Systems through the Sponsored Research Agreement "Research Project for Industry 4.0" and by the European Union through the ADAPTO project, part of the RESTART program, NextGenerationEU PNRR, CUP E63C22002040007, CP PE0000001.

References

[1] Z. Yang, Y. Cui, B. Li, Y. Liu, Y. Xu, Software-Defined Wide Area network (SD-WAN): Architecture, advances and opportunities, in: 2019 28th International Conference on Computer Communication and Networks, ICCCN, 2019, pp. 1–9, http://dx.doi.org/10.1109/ICCCN.2019.8847124.

[2] R. Mohammadi, S. Akleylek, A. Ghaffari, A. Shirmarz, Taxonomy of traffic engineering mechanisms in software-defined networks: A survey, Telecommun. Syst. (2022) 1–28.

[3] A. Navarro, R. Canonico, A. Botta, Software defined wide area networks: Current challenges and future perspectives, in: 2023 IEEE 9th International Conference on Network Softwarization, NetSoft, 2023, pp. 350–353, http://dx.doi.org/10.1109/NetSoft57336.2023.10175458.

[4] L. Borgianni, S. Troia, D. Adami, G. Maier, S. Giordano, From MPLS to SD-WAN to ensure QoS and QoE in cloud-based applications, in: 2023 IEEE 9th International Conference on Network Softwarization, NetSoft, IEEE, 2023, pp. 366–369.

[5] D. Espinel Sarmiento, A. Lebre, L. Nussbaum, A. Chari, Decentralized SDN control plane for a distributed cloud-edge infrastructure: A survey, IEEE Commun. Surv. Tutor. 23 (1) (2021) 256–281, http://dx.doi.org/10.1109/COMST.2021.3050297.

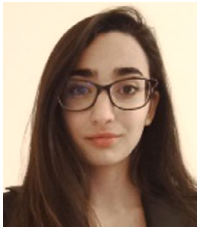
[6] A. Botta, R. Canonico, A. Navarro, G. Stanco, G. Ventre, Scalable reinforcement learning for dynamic overlay selection in SD-WANs, in: 2023 IFIP Networking Conference, IFIP Networking, 2023, pp. 1–9, http://dx.doi.org/10.23919/IFIPNetworking57963.2023.10186399.
[7] D.Z. Tootaghaj, F. Ahmed, P. Sharma, M. Yannakakis, Homa: An efficient topology and route management approach in SD-WAN overlays, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications, IEEE, 2020, pp. 2351–2360.
[8] Z. Duliński, R. Stankiewicz, G. Rzym, P. Wydrych, Dynamic traffic management for SD-WAN inter-cloud communication, IEEE J. Sel. Areas Commun. 38 (7) (2020) 1335–1351.
[9] S. Troia, L.M.M. Zorello, A.J. Maralit, G. Maier, SD-WAN: An open-source implementation for enterprise networking services, in: 2020 22nd International Conference on Transparent Optical Networks, ICTON, 2020, pp. 1–4, http://dx.doi.org/10.1109/ICTON51198.2020.9203058.
[10] A.Y. Kamri, P.T.A. Quang, N. Huin, J. Leguay, Constrained policy optimization for load balancing, in: 2021 17th International Conference on the Design of Reliable Communication Networks, DRCN, IEEE, 2021, pp. 1–6.
[11] S. Troia, F. Sapienza, L. Varé, G. Maier, On Deep Reinforcement Learning for Traffic Engineering in SD-WAN, IEEE J. Sel. Areas Commun. 39 (7) (2021) 2198–2212, http://dx.doi.org/10.1109/JSAC.2020.3041385.
[12] Y. Zhang, J. Tourrilhes, Z.-L. Zhang, P. Sharma, Improving sd-wan resilience: From vertical handoff to wan-aware MPTCP, IEEE Trans. Netw. Serv. Manag. 18 (1) (2021) 347–361.
[13] A. Botta, R. Canonico, A. Navarro, S. Ruggiero, G. Ventre, AI-enabled SD-WAN: The case of reinforcement learning, in: 2022 IEEE Latin-American Conference on Communications, LATINCOM, 2022, pp. 1–6, http://dx.doi.org/10.1109/LATINCOM56090.2022.10000667.
[14] P.T. Anh Quang, S. Martin, J. Leguay, X. Gong, X. Huiying, Intent-based routing policy optimization in SD-WAN, in: ICC 2022 - IEEE International Conference on Communications, 2022, pp. 4914–4919, http://dx.doi.org/10.1109/ICC45855.2022.9838293.
[15] S. Troia, M. Mazzara, M. Savi, L.M.M. Zorello, G. Maier, Resilience of delay-sensitive services with transport-layer monitoring in SD-WAN, IEEE Trans. Netw. Serv. Manag. 19 (3) (2022) 2652–2663, http://dx.doi.org/10.1109/TNSM.2022.3191943.
[16] H. Fawaz, J. Lesca, P.T.A. Quang, J. Leguay, D. Zeghlache, P. Medagliani, Graph convolutional reinforcement learning for collaborative queuing agents, 2022, arXiv preprint arXiv:2205.12009.
[17] Y. Zhang, H. Zhang, P. Cong, W. Wang, K. Xu, Grandet: Cost-aware traffic scheduling without prior knowledge in SD-WAN, in: 2023 IEEE/ACM 31st International Symposium on Quality of Service, IWQoS, IEEE, 2023, pp. 1–10.
[18] P.T.A. Quang, J. Leguay, X. Gong, X. Huiying, Global QoS policy optimization in SD-WAN, in: 2023 IEEE 9th International Conference on Network Softwarization, NetSoft, 2023, pp. 202–206, http://dx.doi.org/10.1109/NetSoft57336.2023.10175407.
[19] H. Fawaz, O. Houidi, D. Zeghlache, J. Lesca, P.T.A. Quang, J. Leguay, P. Medagliani, Graph convolutional reinforcement learning for load balancing and smart queuing, in: 2023 IFIP Networking Conference, IFIP Networking, 2023, pp. 1–9, http://dx.doi.org/10.23919/IFIPNetworking57963.2023.10186430.
[20] S. Dou, L. Qi, C. Yao, Z. Guo, Exploring the impact of critical programmability on controller placement for software-Defined wide area networks, IEEE/ACM Trans. Netw. (2023) 1–14, http://dx.doi.org/10.1109/TNET.2023.3252639.
[21] M.A. Ouamri, G. Barb, D. Singh, F. Alexa, Load balancing optimization in software-defined wide area networking (SD-WAN) using deep reinforcement learning, in: 2022 International Symposium on Electronics and Telecommunications, ISETC, IEEE, 2022, pp. 1–6.
[22] D.E. Sarmiento, A. Lebre, L. Nussbaum, A. Chari, Decentralized SDN control plane for a distributed cloud-edge infrastructure: A survey, IEEE Commun. Surv. Tutor. 23 (1) (2021) 256–281.
[23] K. Phemius, M. Bouet, J. Leguay, Disco: Distributed multi-domain sdn controllers, in: 2014 IEEE Network Operations and Management Symposium, NOMS, IEEE, 2014, pp. 1–4.
[24] S. Ahmad, A.H. Mir, Scalability, consistency, reliability and security in SDN controllers: A survey of diverse SDN controllers, J. Netw. Syst. Manage. 29 (2021) 1–59.
[25] W. Mao, L. Yang, K. Zhang, T. Basar, On improving model-free algorithms for decentralized multi-agent reinforcement learning, in: International Conference on Machine Learning, PMLR, 2022, pp. 15007–15049.
[26] S. Sarkar, An algorithm for adversary aware decentralized networked MARL, 2023, arXiv preprint arXiv:2305.05573.
[27] K. Zhang, Z. Yang, H. Liu, T. Zhang, T. Basar, Fully decentralized multi-agent reinforcement learning with networked agents, in: International Conference on Machine Learning, PMLR, 2018, pp. 5872–5881.
[28] Y. Yi, G. Li, Y. Wang, Z. Lu, Learning to share in multi-agent reinforcement learning, 2021, arXiv preprint arXiv:2112.08702.
[29] P. Segeč, M. Moravčik, J. Uratmová, J. Papán, O. Yeremenko, SD-WAN-architecture, functions and benefits, in: 2020 18th International Conference on Emerging ELearning Technologies and Applications, ICETA, IEEE, 2020, pp. 593–599.
[30] S. Troia, L.M. Moreira Zorello, G. Maier, SD-WAN: How the control of the network can be shifted from core to edge, in: 2021 International Conference on Optical Network Design and Modeling, ONDM, 2021, pp. 1–3, http://dx.doi.org/10.23919/ONDM51796.2021.9492375.
[31] C. Scarpitta, G. Sidoretti, A. Mayer, S. Salsano, A. Abdelsalam, C. Filsfils, High performance delay monitoring for SRv6 based SD-WANs, 2022, arXiv preprint arXiv:2212.12627.
[32] R.K. Rangan, Trends in SD-WAN and SDN, CSI Trans. ICT 8 (1) (2020) 21–27.
[33] Y. Zhao, L. Iannone, M. Riguidel, On the performance of SDN controllers: A reality check, in: 2015 IEEE Conference on Network Function Virtualization and Software Defined Network, NFV-SDN, IEEE, 2015, pp. 79–85.
[34] Y. Xiao, J. Liu, J. Wu, N. Ansari, Leveraging deep reinforcement learning for traffic engineering: A survey, IEEE Commun. Surv. Tutor. 23 (4) (2021) 2064–2097.
[35] N. Geng, T. Lan, V. Aggarwal, Y. Yang, M. Xu, A multi-agent reinforcement learning perspective on distributed traffic engineering, in: 2020 IEEE 28th International Conference on Network Protocols, ICNP, IEEE, 2020, pp. 1–11.
[36] A. Botta, A. Dainotti, A. Pescapè, A tool for the generation of realistic network workload for emerging networking scenarios, Comput. Netw. 56 (15) (2012) 3531–3547.



Alessio Botta received the M.S. degree in Telecommunications Engineering and the Ph.D. degree in Information Technology and Electrical Engineering from the University of Napoli 'Federico II'. He is currently an Associate Professor at the University of Napoli `Federico II'. He has co-authored over 100 international journal and conference publications, and has chaired international conferences and workshops.



Roberto Canonico is Associate Professor at the University of Napoli 'Federico II' since 2005. He received the Laurea degree (cum laude) in Electronic Engineering from the University of Napoli 'Federico II' in 1995, and a Ph.D. in Computer Engineering from the same University in 2000. In 2000, he was a visiting Research Associate at Lancaster University, UK. His current research interests include Software Defined Networking, applications of AI to networking, green networking, resource management in cloud-native network architectures.



Annalisa Navarro obtained a Master's degree cum laude in Computer Engineering at the University of Napoli 'Federico II' in 2022. She is currently a Ph.D. student of Information Technology and Electrical Engineering at the same university after working for six months as a Research Scholar. Her current research focuses on SD-WAN performance optimization using Machine Learning techniques.



Giovanni Stanco received the M.Sc. degree in Telecommunications Engineering from the University of Napoli 'Federico II' in 2019, and the Ph.D. in Information Technology and Electrical Engineering from the same University in 2023. His current research interests include networking in IoT and Cyber–Physical Systems, and Software Defined Networking.



Giorgio Ventre is Full Professor of Computer Networks at the Department of Information Technology and Electrical Engineering of the University of Napoli 'Federico II'. He owns a Laurea Degree in Electronic Engineering and a Ph.D. in Computer Engineering, both from the University of Napoli 'Federico II'. He is currently the Scientific Director of the Apple Developer Academy in Napoli. He is also member of several Scientific Committees and seats in the Board of public and private entities.